

# Requisiti del software

## Ingegneria dei requisiti

Lo scopo dell'**ingegneria dei requisiti** (RE, *Requirements Engineering*) è quello di stilare e mantenere il documento di **analisi dei requisiti di sistema** (SRS, *Software Requirements Specification*). Il documento dei requisiti di sistema è una dichiarazione ufficiale dei servizi forniti da un sistema e dei suoi vincoli operativi.

Vi sono due livelli di descrizione:

- i **requisiti utente** dichiarano in linguaggio naturale o attraverso diagrammi quali servizi il sistema dovrebbe fornire;
- i **requisiti di sistema** definiscono i servizi del sistema in modo estremamente dettagliato e preciso.

Oltre questo livello di descrizione, generalmente i requisiti sono suddivisi in:

- **requisiti funzionali**: descrivono le *funzionalità* che il sistema deve fornire (per esempio: *il sistema dà la possibilità di registrare un utente e successivamente eliminarlo....*)
- **requisiti non funzionali**: descrivono *vincoli* sui servizi e sulle funzionalità offerte (per esempio: *il sistema garantisce al massimo un numero di registrazioni pari a 1000 utenti....*). Possono riferirsi a proprietà del sistema come *affidabilità*, *tempi di risposta* e *occupazione di memoria*. Sono spesso più critici di quelli funzionali in quanto la mancata gestione di uno di questi vincoli potrebbe potenzialmente causare l'inutilizzabilità del prodotto finale. I principali tipi di requisiti non funzionali sono:
  - *requisiti del prodotto*: specificano il comportamento del prodotto
  - *requisiti organizzativi*: specificano le procedure di organizzazione tra cliente e team di sviluppatori
  - *requisiti esterni*: requisiti provenienti da fattori esterni al sistema
- **requisiti di dominio**: descrivono requisiti che derivano dal *dominio applicativo* e possono essere funzionali o non funzionali.

## Requisiti utente

I requisiti utente di un sistema descrivono requisiti funzionali e non funzionali in modo comprensibile per gli utenti del sistema senza un'approfondita conoscenza tecnica. Dovrebbero essere scritte in linguaggio semplice, attraverso tabelle, diagrammi e schemi intuitivi. Tuttavia è risaputo come il linguaggio naturale sia di per se ambiguo, pertanto molto spesso i requisiti utente sono soggetti a spiacevoli incomprensioni; per minimizzare questo, è possibile seguire delle linee guida:

- utilizzare un formato standard (inteso come formattazione del documento per risaltare i concetti e le funzionalità)
- usare il linguaggio in maniera coerente (per esempio utilizzando *"deve"* per i requisiti obbligatori, o *"dovrebbe"* per quelli desiderati)
- utilizzare un dizionario dei termini che specifichi l'utilizzo di alcuni termini che altrimenti potrebbero assumere un significato ambiguo

## Requisiti di sistema

I requisiti di sistema sono versioni espanse di quelli utente utilizzati dagli ingegneri del software come base di partenza della fase di progettazione. Possono essere utilizzati come parte del contratto e dovrebbero dunque risultare dettagliati, coerenti e precisi. Per questo necessitano di essere scritti attraverso notazioni dettagliate come il *linguaggio naturale strutturato*, *modelli grafici* o gli *use-case* (casi d'uso).

### ***Specifiche in linguaggio strutturato***

Le notazioni in linguaggio strutturato limitano la terminologia che può essere usata nella descrizione dei requisiti e utilizzano modelli predefiniti. Ci sono tre tipi di possibili notazioni:

- specifiche basate sui form
- specifiche tabulari
- modelli grafici

### ***Processi di ingegneria dei requisiti***

L'ingegneria dei requisiti suddivide il processo generale di stesura dei requisiti di sistema in 4 sotto-processi di alto livello:

- studio di fattibilità
- deduzione e analisi dei requisiti
- convalida dei requisiti
- manutenzione dei requisiti

### ***Studio di fattibilità***

Il processo di ingegneria dei requisiti inizia con uno studio di fattibilità, che consiste nello stabilire perloppunto se il sistema proposto è *fattibile* o no. In sostanza l'output di questo studio è un report che risponde alle domande: il sistema contribuirà agli obiettivi? Può essere implementato con le tecnologie attuali, nel rispetto dei costi prefissati e dei vincoli temporali? Può essere integrato ad altri sistemi già in commercio? In questa fase si effettua una profonda consultazione delle sorgenti di informazione quali manager dei dipartimenti, ingegneri, esperti business e utenti finali.

### ***Deduzione e analisi dei requisiti***

In questa fase gli ingegneri lavorano assieme ai vari stakeholders del sistema per scoprire informazioni sul dominio applicativo. Potrebbe essere una comunicazione per niente facile in quanto:

- gli stakeholders spesso non sanno cosa vogliono
- gli stakeholders esprimono i concetti in linguaggio naturale che può risultare impreciso
- diversi stakeholders hanno requisiti contrastanti
- l'ambiente economico è totalmente dinamico, per cui è possibile assistere all'emersione di nuovi stakeholders e quindi di nuovi requisiti

Tutte queste eventualità rendono il documento dei requisiti e l'intero processo di stesura, un'operazione particolarmente dinamica, che prevede una continua correzione e modifica dei requisiti.

Prima di descrivere il processo di convalida, segue una descrizione dei linguaggi strutturati maggiormente utilizzati nella stesura del documento dei requisiti.

### ***Punti di vista***

I punti di vista sono delle notazioni che consentono di rappresentare i requisiti sotto l'aspetto delle differenti prospettive degli stakeholders. Per farlo vengono classificati gli stakeholders in rispettivi punti di vista e poi classificati in:

- *punti di vista interattivi*: rappresentano le persone che interagiscono direttamente col sistema
- *punti di vista indiretti*: rappresentano le persone che non usano direttamente il sistema ma ne influenzano i requisiti
- *punti di vista di dominio*: rappresentano vincoli di dominio che influenzano i requisiti di sistema.

### **Scenari**

Gli scenari sono esempi di interazione che si riferiscono a esempi di vita reale e che si occupano di uno o più interazioni possibili. Possono essere testi scritti o diagrammi, screen shots o in alternativa use-case.

### **Casi d'uso (use-case)**

Sono una tecnica basata su scenari per la deduzione dei requisiti durante la fase di analisi. La notazione maggiormente in uso è quella UML attraverso il diagramma dei casi d'uso che sviluppa lo scenario con un livello di dettaglio maggiore.

### **Convalida dei requisiti**

La convalida dei requisiti consiste nel mostrare che essi definiscono veramente il sistema che il cliente vuole. È una fase importante in quanto gli errori nel documento possono portare ad un costo di lavorazione molto alto se vengono scoperti durante la fase di progettazione o ancora peggio in quella di validazione. Durante questa fase è possibile effettuare una serie di tecniche di validazione:

- revisione dei requisiti
- prototipizzazione: viene mostrato un modello ridotto eseguibile del sistema agli utenti finali
- test-case: vengono generati dei casi di test per rivelare eventuali problemi nei requisiti.

### **Manutenzione dei requisiti**

La gestione dei requisiti è il processo di comprensione e di controllo dei cambiamenti ai requisiti di sistema. Per fare questo è necessario mantenere alcuni collegamenti tra i requisiti ed il processo di sviluppo del software e stabilire un processo formale per proporre cambiamenti. La *tracciabilità* è la proprietà di una specifica di trovare il requisito relativo nel documento dei requisiti. Vi sono varie tipologie di tracciabilità:

- *tracciabilità della sorgente*: collegamenti tra i requisiti e gli stakeholders che li hanno indicati
- *tracciabilità dei requisiti*: collegamenti tra i requisiti stessi all'interno del documento; così facendo si crea una dipendenza tra i requisiti e in caso di cambiamento di un requisito sarebbero valutati gli impatti sugli altri requisiti annessi
- *tracciabilità del progetto*: collegamenti tra i requisiti e i moduli del progetto in cui sono stati implementati; così facendo vengono valutati gli impatti di eventuali cambiamenti direttamente sull'implementazione del sistema.

# SOFTWARE DESIGN

Il processo di “Software Design”, che si sviluppa in varie fasi, è la realizzazione di una struttura operativa che si fonda sulle specifiche e sui requisiti richiesti dall’utente; la traduzione di tale struttura in un programma eseguibile è detta implementazione.

Queste due differenti attività, tuttavia, sono molte collegate fra loro e possono essere intervallate.

## Architettura del Software

La progettazione architeturale di un software è l’identificazione dei suoi sotto-sistemi e la specifica delle strutture di controllo e comunicazione. L’intera architettura è normalmente presentata attraverso l’uso di un diagramma a blocchi, utile per la comunicazione con gli stakeholders ma non propriamente adatto per la rappresentazione delle relazioni fra i vari componenti o delle proprietà specifiche dei sotto-sistemi. Diversi modelli di rappresentazione, tuttavia, possono essere sviluppati durante questa fase.

Caratteristiche principali che ogni architettura dovrebbe possedere sono le seguenti:

- Performance (uso di grandi componenti al posto di componenti piccoli e specifici)
- Sicurezza (uso di una struttura con più livelli con le fasi critiche inserite nei livelli più bassi)
- Disponibilità (inclusione di meccanismi per la tolleranza all’errore)
- Manutenibilità (uso di componenti rimpiazzabili)

### **Modelli Architettureali:**

- 1) Modello Statico - mostra la maggior parte dei componenti di sistema
- 2) Modello Dinamico – mostra la struttura dei processi di sistema
- 3) Modello a Interfaccia – mostra e definisce le interfacce dei sotto-sistemi
- 4) Modello a Relazioni – mostra le relazioni che intercorrono tra i vari sotto-sistemi
- 5) Modello di Distribuzione – mostra la distribuzione dei sotto-sistemi sui vari calcolatori

## Organizzazione del Sistema

Ci sono tre diversi stili per organizzare un sistema informatico:

- 1) Stile con repository centrale condiviso
- 2) Stile con server e servizi condivisi
- 3) Stile con macchina a più strati o livelli

**REPOSITORY CENTRALE:** largamente usato quando una vasta mole di dati deve essere condivisa, i vari sotto-sistemi scambiano dati fra loro: tutti i dati che devono essere condivisi all’interno del sistema sono conservati nel repository centrale.

**PREGI:** modo efficiente di scambiare dati; i sotto-sistemi non devono preoccuparsi della sicurezza e della gestione dei dati;

**DIFETTI:** difficile da ampliare in modo efficiente; i sotto-sistemi devono accettare il modello di gestione dei dati proprio del repository centrale.

**CLIENT-SERVER:** questo tipo di modello è basato sulla distribuzione e il processamento dei dati su una più ampia rete di componenti. Insieme di server, che offrono un servizio, sono chiamati da insieme di client che richiedono tale servizio. I clients per ricevere un servizio devono essere a conoscenza dei server che lo offrono, mentre per i server non è necessario conoscere i client. Sia server che client sono basati su processori logici. Fondamentale e necessario è l'utilizzo della rete.

**PREGI:** la distribuzione dei dati è molto semplificata; facile aggiungere nuovi server e/o aggiornarli

**DIFETTI:** i vari sotto-sistemi usano modelli diversi di gestione dei dati perciò lo scambio di essi potrebbe risultare inefficiente; gestione del servizio offerto in ogni server (ridondante).

**MACCHINA A STRATI:** consiste nell'organizzare tutto il sistema tramite un insieme di livelli, ognuno dei quali offre un determinato servizio; ciò permette il supporto di uno sviluppo incrementale dei sotto-sistemi (in caso di guasto o aggiornamento solo il livello specifico viene modificato). Nelle macchine a strati sono presenti solitamente tre livelli principali: un livello di presentazione (che si occupa di interfacciarsi con l'utente), un livello applicativo (che si occupa delle funzioni specifiche del programma) e un livello di gestione di dati (che si occupa della completa gestione delle basi di dati). Strutturare un sistema in questo modo, tuttavia, può risultare abbastanza complicato.

## **Moduli & Sotto-sistemi**

Un sotto-sistema è un sistema in proprio le cui operazioni sono indipendenti dagli altri sotto-sistemi. Un modulo, invece, è un componente di sistema che offre servizi agli altri componenti, tuttavia non viene considerato come un sistema separato.

Solitamente i sotto-sistemi vengono decomposti in componenti e ciò può avvenire tramite due modelli: il modello a oggetti e il modello a pipeline.

**MODELLO A OGGETTI:** basato sulla decomposizione del sistema in molteplici oggetti, ognuno con propri attributi e operazioni, che ne determinano l'interfaccia. L'uso di oggetti è molto conveniente in quanto possono essere modificati individualmente e rispecchiano il mondo reale. Unico difetto è che entità molto complesse potrebbero essere difficili da rappresentare come oggetti.

**MODELLO A PIPELINE:** basato su processi di trasformazione dell'input con risultato l'output. Tale trasformazione è spesso effettuata in maniera sequenziale. I pregi del modello sono vari: facilità di aggiungere nuove trasformazioni, facilità di implementazione e intuitiva organizzazione per la comunicazione con gli stakeholders. Difetto principale di questo modello è la sua non adattabilità per sistemi interattivi e un formato comune per il trasferimento dei dati lungo la pipeline.

## **Stili di Controllo**

Riguardano il flusso di controllo fra i vari sotto-sistemi. Ne esistono due tipi principali:

- Controllo centralizzato: un solo sotto-sistema si occupa del controllo di tutti gli altri.
- Controllo events-based: ogni sotto-sistema può rispondere ad eventi generati esternamente.

**CONTROLLO CENTRALIZZATO:** un solo sotto-sistema si occupa del controllo e della gestione degli altri sotto-sistemi. Esistono due approcci diversi:

- 1) Modello call-return: dove il controllo inizia sulla cima di una routine gerarchica e man mano si muove verso il basso. Applicabile a sistemi sequenziali.
- 2) Modello manageriale: un componente di sistema controlla e coordina tutti gli altri processi di sistema. Applicabile anche a sistemi concorrenti.

**CONTROLLO EVENTS-BASED:** basato sulla generazione e sul processamento di eventi esterni, processati da uno dei sotto-sistemi. Esistono due modelli principali:

- 1) Modello broadcast: dove un evento è mandato in broadcast in tutto il sistema e ogni sotto-sistema può gestire tale evento; ogni sotto-sistema, infatti, se interessato ad un particolare evento, può fare richiesta per la sua gestione. I sotto-sistemi, tuttavia, non sono in grado di sapere a priori quando questa gestione avverrà.
- 2) Modello interrupt-based: usato molto nei sistemi reali dove il tempo di risposta ad un evento è fondamentale. Ci sono vari tipi di interruzioni e gestori per ognuna di esse. A ognuna di esse è associata una locazione di memoria, a cui verrà indirizzato il gestore specifico. Permette un tempo di risposta molto rapido ma la sua progettazione è molto complessa.

### ***Tipi di Sistema***

- 1) Sistemi personali: non distribuiti e progettati per funzionare su PC o workstation
- 2) Sistemi embedded: progettati per funzionare su un singolo processore o un insieme di essi
- 3) Sistemi distribuiti: progettati per funzionare su un gruppo cooperante di processori connessi tra loro da una rete

### ***ARCHITETTURE A SISTEMI DISTRIBUITI***

Il processamento dell'informazione è distribuito su diversi computer piuttosto che concentrata su un singolo e unico calcolatore. Questo tipo di sistema permette la condivisione delle risorse hardware e software, il processamento di dati in maniera concorrente, grande tolleranza agli errori e scalabilità, vista la possibilità di aggiungere in modo semplificato nuovo hardware per l'incremento delle prestazioni del sistema. Difetti di tale sistema sono la scarsa sicurezza e semplicità di progettazione rispetto al sistema centralizzato e soprattutto l'imprevedibilità delle risposte, molto dipendente dall'organizzazione di sistema e dal carico presente sulla rete.

### ***ARCHITETTURE CLIENT-SERVER***

- 1) Modello a client leggero, in cui tutto il processamento dei dati e l'applicazione viene gestita solamente lato server e quindi il compito del client è solamente la presentazione dei risultati. Difetto principale: sia il server sia la rete sono soggette ad un grosso carico
- 2) Modello a client grasso, in cui il server si occupa solamente della gestione dei dati: tutta la logica dell'applicazione viene gestita dal lato client, così da alleggerire il carico di lavoro del server. Difetto principale: molto più complicato del modello a client leggero.

### ***ARCHITETTURE A OGGETTI DISTRIBUITI***

In questo tipo di architettura non c'è alcuna distinzione fra client e server e ogni entità distribuita è un oggetto che può mettere a disposizione o ricevere servizi. La comunicazione fra i vari oggetti avviene attraverso un sistema di intermezzo chiamato Broker, un esempio di questo è CORBA. Vantaggi di tale architettura sono la sua grande flessibilità e scalabilità.

## **ARCHITETTURE P2P**

I sistemi P2P sono decentralizzati e la computazione dei dati può essere gestita da uno dei nodi nella rete. L'intero sistema è progettato per sfruttare tutto il potere computazionale e lo spazio dei computer connessi alla rete, che possono essere appoggiati, in alcuni casi, da un "discovery server".

## **PROGETTAZIONE ORIENTATA AGLI OGGETTI**

Si articola in tre fasi principali:

- 1) OOA: sviluppo di un modello del dominio dell'applicazione (analisi)
- 2) OOD: sviluppo di un modello orientato a oggetti per implementare i requisiti richiesti
- 3) OOP: realizzare un modello orientato a oggetti usando un linguaggio di programmazione orientato a oggetti come Java o C++

Gli oggetti sono astrazioni delle entità del mondo reale. Sono indipendenti gli uni dagli altri e permettono di incapsulare al loro interno informazioni per poi esporle tramite appropriate funzioni. In questo tipo di approccio le aree di dati condivisi sono eliminate e gli oggetti comunicano fra loro tramite passaggio di messaggi. Gli oggetti possono, inoltre, essere distribuiti ed essere processati in maniera sequenziale o parallela.

Vantaggi di tale approccio sono i seguenti:

- Facile manutenzione: gli oggetti, infatti, possono essere visti come entità a sé stanti
- Riutilizzabilità: gli oggetti sono componenti potenzialmente riutilizzabili
- Metafore naturali: gli oggetti possono essere mappati con le entità

## **PROCESSO DI PROGETTAZIONE**

Il processo di progettazione orientata a oggetti di un sistema comporta molto lavoro sia durante lo sviluppo che la manutenzione. Questo tipo di processo si articola in varie fasi:

- **Definizione del contesto e dei modelli di uso del sistema:** consiste nello sviluppo di un modello statico che descrive l'ambiente in cui il sistema si trova a operare e di un modello dinamico che descrive come tale sistema interagisca con l'ambiente circostante;
- **Progettazione dell'architettura del sistema;**
- **Identificazione dei principali oggetti del sistema:** è la parte più complessa di tutta la progettazione orientata a oggetti, poiché non esiste alcuna formula specifica per l'identificazione dei vari oggetti che comporranno il sistema.  
Durante questa particolare fase si può ricorrere all'uso di vari modelli:

- 1) *Modello dei sotto-sistemi* il cui compito è mostrare il raggruppamento logico dei vari oggetti in sotto-sistemi coerenti.
- 2) *Modello a sequenza:* il cui compito è mostrare la sequenza delle varie interazioni fra i vari oggetti
- 3) *Modello a stati macchina* il cui compito è mostrare come i singoli oggetti cambino il loro stato in risposta a determinati eventi

- **Specificazione delle interfacce degli oggetti.**





## Processi software

Un processo software è un insieme strutturato di attività richieste per lo sviluppo o l'evoluzione di un software.

Tra le attività generiche comuni a tutti i processi software ricordiamo:

- Specifiche
- Progettazione
- Validazione
- Evoluzione.

Un modello di un processo software è una rappresentazione astratta di un generico processo. In particolare, esso presenta una descrizione del processo da un particolare punto di vista.

Tre i modelli generici di un processo software:

- ✓ Modello a cascata
- ✓ Sviluppo evolutivo
- ✓ Ingegneria del software basata sui componenti

### ➤ MODELLO A CASCATA

In tale modello, le fasi di specifiche e sviluppo sono delle fasi distinte e separate.

In particolare, le fasi di un modello a cascata sono:

- Analisi e definizione dei requisiti
- Progettazione software
- Implementazione
- Integrazione e sistema di testing
- Manutenzione

Il principale svantaggio nell'utilizzo di tale modello è la difficoltà di cambiamenti dopo il processo. In effetti, una conseguenza del modello è che ogni fase deve essere completata prima di passare alla fase successiva.

Il modello a cascata presenta vari problemi, come l'inflessibilità di suddividere il progetto in fasi distinte che costituisce una difficoltà nel cambiamento dei requisiti del cliente. Il modello a cascata è adatto quando i requisiti sono ben capiti e i cambiamenti dei requisiti saranno limitati durante la fase di progettazione e non oltre. Il modello a cascata è molto utilizzato per progetti molto grandi dove il sistema è sviluppato in molte aree.

### ➤ SVILUPPO EVOLUTIVO

Le fasi di specifiche, sviluppo e validazione possono avvenire insieme, ovvero si ripetono una con l'altra.

Abbiamo due tipi di sviluppo evolutivo:

-**sviluppo esplorativo**, il cui obiettivo è lavorare con il cliente e evolvere in un sistema finale da alcune specifiche iniziali. Quindi iniziare con poche specifiche ben capite e aggiungere nuove caratteristiche proposte dal cliente.

-**prototipo Usa e getta**, il cui obiettivo è capire i requisiti del sistema; quindi iniziare con i requisiti poco chiari per chiarire che cosa realmente bisogna fare.

Nello sviluppo evolutivo sono presenti alcuni problemi, come la scarsa visibilità del processo, oppure potrebbe essere generato un software mal strutturato.

Tale modello può essere applicato per sistemi interattivi di piccola e media dimensione, oppure essere parte dello sviluppo di un grande sistema.

### ➤ Ingegneria del software basata sui componenti

Modello che si basa sul riutilizzo sistematico. Ovvero generare un sistema che è integrato da componenti esistenti o da COTS.

Le fasi del processo sono:

- Analisi dei componenti
- Modificazione dei requisiti
- Progettazione con riutilizzo
- Sviluppo e integrazione.

Processi di iterazione

I requisiti evolvono sempre nel corso di un progetto.

L'iterazione può essere applicata per qualunque modello generico di processo software.

In particolare, ci sono due approcci di iterazione:

- Consegna incrementale
- Sviluppo spirale

### CONSEGNA INCREMENTALE

La fase di sviluppo e di consegna sono suddivise in incrementi; ogni incremento consegna parte dei requisiti funzionali.

Una volta che lo sviluppo di un incremento è iniziato, i requisiti sono bloccati anche se i requisiti per gli incrementi successivi possono continuare ad evolvere.

Sono presenti vari vantaggi della consegna incrementale, per esempio, il cliente valuta ogni consegna e quindi la funzionalità del sistema è disponibile da subito. Ogni incremento fornisce un prototipo che aiuta a sviluppare nuovi requisiti o migliorarli

per i prossimi incrementi. Ovviamente così facendo si ha un rischio minore di fallire l'intero progetto.

### SVILUPPO SPIRALE

Il processo è rappresentato come una spirale piuttosto che rappresentato come una sequenza di attività.

Ogni ciclo della spirale rappresenta una fase del processo.

I rischi sono valutati e risolti durante tutto il processo.

### Software CASE

Un CASE è un software sviluppato per il supporto delle attività dei processi software, in particolare per l'attività di sviluppo ed evoluzione.

La tecnologia CASE ha portato a miglioramenti significativi nei processi di software. Dopotutto, ricordiamo che l'ingegneria del software richiede creatività e quindi l'uso di tale tecnologia CASE non è consente di progettare con creatività. In effetti l'ingegneria del software è un team di persone che interagiscono tra di loro per la realizzazione comune del progetto. Quindi la tecnologia CASE non supporta realmente gli aspetti umani e di interazione tra umani che definiscono caratteristiche importante per lo sviluppo software.

Integrazione CASE:

- Tools: utilizzati per il supporto di compiti individuali come Text Editing.
- Workbenches: più tools messi insieme, quindi definiscono fasi di processi.
- Environments: più workbenches messi insieme, supporto per l'intero processo software.

## PROJECT MANAGEMENT

### SOFTWARE PROJECT MANAGEMENT (Gestione del progetto Del software)

Esistono alcune attività specifiche per garantire, che il software venga consegnato nei tempi previsti, e che sia in conformità con i requisiti previsti dall'organizzazione destinata a sviluppare il software!

### MANAGEMENT ACTIVITIES (Attività di gestione)

Queste Attività si suddividono in:

- Scrittura di proposte.
- Pianificazione e Programmazione
- Costi per la realizzazione del Progetto
- Monitoraggio del progetto e delle revisioni.
- Selezione e valutazione del personale
- Scrittura di report e presentazioni

### PROJECT PLANNING (Pianificazione del Progetto)

La pianificazione, tra le attività di gestione, è quella che richiede maggior tempo.

In questa fase si susseguono continue attività a partire dall'idea iniziale fino al rilascio del sistema, per questo bisognerà rivedere regolarmente i piani Pianificati precedentemente in quanto sono soggetti a modifiche e vi saranno disponibili nuove informazioni.

Differenti tipi di piani potrebbero essere sviluppati per dare supporto al progetto software principale.

### THE PROJECT PLAN (Il piano del progetto)

Il piano del progetto Software Principale dovrà essere redatto in modo tale che si occuperà di:

- Gestire Le risorse disponibili per il progetto, suddividere la ripartizione del lavoro, e organizzare un programma per il lavoro.

### PROJECT PLAN STRUCTURE (Struttura del piano di progetto)

Il piano del progetto software principale sarà suddiviso in modo tale da comprendere le seguenti parti.

Avrà un Introduzione, Indicherà l'organizzazione del progetto, vi sarà un analisi dei rischi, indicherà le risorse hardware e software richieste, come sarà pianificato il progetto, e infine i meccanismi di monitoraggio e di reporting.

### ATTIVITA' ORGANIZZATIVE

Un progetto sarò organizzato in più attività, ciascuna delle quali dovrà essere organizzata in modo da produrre risultati tangibili al fine di giudicarne il progresso.

Le Milestones sono il punto finale di un processo di attività.

Le Deliverables sono risultati di progetto consegnati al cliente.

Il Processo a Cascata sposa perfettamente il concetto di indicare le Milestones alla fine di ogni attività, proprio per la struttura con cui esso viene svolto!

## PROJECT SCHEDULING (PROGRAMMAZIONE DEL PROGETTO)

Il Progetto verrà suddiviso in più Parti ognuna delle quali indicherà un'attività, e per ognuna di esse verrà stimato il tempo e le risorse necessarie per completarla. Bisognerà ridurre al minimo le dipendenze delle attività per evitare ritardi causati da attività in attesa di un'altra che ancora non è giunta al termine, per poter essere completata.

## SOFTWARE COST ESTIMATION (Stima dei costi del software)

La stima del costo del progetto e la programmazione sono attività di gestione interlacciate. Per la stima finale del costo del software bisogna porsi delle domande, alcune delle quali per esempio: Quanta fatica è necessaria per completare un'attività? Quanto tempo è necessario per completare un'attività? Qual è il costo totale di un'attività?

## SOFTWARE COST COMPONENTS (Componenti di costo di un software)

Nella stima finale del costo di un software, bisogna tener conto dei costi relativi agli hardware e ai software che si utilizzeranno in fase di progettazione, dei costi dei viaggi e di aggiornamento degli impiegati, dei costi detti "costi di sforzo" che ricoprono la maggior parte dei costi di un progetto poiché in queste voci troveremo i salari degli ingegneri coinvolti nella realizzazione del progetto, i costi sociali e assicurativi, i "costi sforzo" comprenderanno anche le voci relative ai costi di spese generali, come quelle di costruzione, relative al cibo alle luci, al networking, comunicazione, e altri costi di servizi comune! Non esiste una relazione tra i costi sostenuti in fase di sviluppo e il prezzo finale che sarà a carico del cliente.

## SOFTWARE PRICING FACTORS (Fattori di prezzo del software)

Esistono alcuni Fattori che possono essere utili per determinare il prezzo finale che un software dovrà avere.

**Market Opportunity:** (Opportunità di Mercato), Un organizzazione di sviluppo, può decidere di inserire in commercio il software ad un prezzo basso, poiché vuole entrare in una nuova fascia del mercato software, cosicché accettando un profitto basso può darsi che in futuro si abbia la possibilità di incrementare i profitti.

**Cost Estimate Uncertainty:** (Costo stimato con incertezza) L'Organizzazione di sviluppo del software non è sicura del prezzo che ha stimato per il software, quindi il suo prezzo verrà incrementato poco al di sopra del suo normale profitto.

**Contractual Terms:** (Termini contrattuali) Un cliente può decidere di lasciare agli sviluppatori di conservare il codice sorgente così da poter essere riutilizzato in altri progetti in modo tale che il prezzo di vendita al cliente sia inferiore a quello previsto.

**Requirements Volatility:** (Requisiti Volatilità) Utilizzati se bisogna ottenere un appalto, si effettua una stima generale con contenuti non specifici in modo tale da ottenere un prezzo inferiore, una volta che l'appalto viene aggiudicato il prezzo del software può aumentare in base a determinate specifiche modificate all'interno dei requisiti.

**Financial Health:** Gli sviluppatori sono in difficoltà finanziaria quindi decidono di avere un profitto basso e restare quantomeno in pareggio piuttosto che andare fuori mercato.

## SOFTWARE PRODUCTIVITY (Produttività del Software)

Per ottenere maggiore velocità in fase di sviluppo, gli ingegneri coinvolti nello sviluppo del software oltre a produrre codice, dovranno produrvi la relativa documentazione associata. Fondamentalmente si vuole misurare l'utile in funzionalità a ciò che si produce in un'unità di tempo prefissata.

## PRODUCTIVITY MEASURES (Misure di produttività)

Esistono alcune misure per determinare la produttività del Software:

La Grandezza di una di queste misure è basata su determinati output ottenuti dal processo software, come le linee di codice scritte, le istruzioni, gli oggetti ecc..(LOC Measures)

Oppure Misure correlate alle Funzionalità, basate su una stima delle funzionalità rilasciate dal software, per quanto riguarda le Funzionalità, la misura più conosciuta è quella relativa alle Funzioni-Punto. (Function Point Measures)

## LINE OF CODE(LOC) (Linee di Codice)

Questa misura delle linee di codice è stata introdotta per la prima volta quando i programmi venivano digitati su schede con una riga per ogni carta. Questo modello assume che vi sia una relazione lineare tra le dimensioni del sistema e il volume della documentazione

## PRODUCTIVITY COMPARISON (Confronti di produttività)

Più è basso il livello del linguaggio, più è produttivo il programmatore! Misure di produttività basate su linee di codice suggeriscono che i programmatori che scrivono molto codice sono più produttivi di programmatori che scrivono il codice compatto.

## FUNCTION POINTS (Funzioni Punto)

Basate su una combinazione delle caratteristiche del programma, come:

- Input e/o output esterni
- Iterazioni con l'utente.
- Interfacce esterne.
- File usati dal sistema

A ciascuno di questi punti soprelencati è associato un peso e la Funzione Punto è calcolata moltiplicando ciascuna riga al suo peso specifico e poi sommandole tutte.

$$UFC = \text{Sommatoria} ((\text{numero di elementi nella riga}) \times (\text{peso}))$$

Il Conteggio di queste Funzioni Punto è modificato dalla complessità del progetto.

FPs possono essere usate per fare una stima delle LOC, che in dipendono in media dal numero di LOC per FP per un determinato linguaggio.

## OBJECT POINTS (Oggetti punto)

Sono un'alternativa, chiamati anche APPLICATION POINTS, non sono la stessa cosa come gli oggetti di una classe.

Il numero di OP in un programma sono stimati sul:

- Numero di schermi separati che vengono visualizzati
- Numero di segnalazioni che vengono prodotte dal sistema.

-Numero di moduli che devono essere sviluppati per integrare il codice per il database.  
Gli OP sono più facili da stimare rispetto alle FP, si interessano soltanto di schermi relazione e moduli di linguaggio di programmazione.  
Possono essere stimati facilmente nelle prime fasi del processo di sviluppo, successivamente sarà però difficile stimare il numero di linee di codice in un sistema!

### **FACTORS AFFECTING PRODUCTIVITY** (Fattori che influenzano la produttività)

**Application Domain:** (Dominio Applicativo) La conoscenza del dominio applicativo è essenziale per l'efficacia dello sviluppo del software. Gli ingegneri che lo comprendono sono più produttivi rispetto a chi non riesce a comprendere bene il dominio.

**Process Quality:** (Qualità del processo) Il Processo di sviluppo utilizzato può avere effetti significativi sulla produttività.

**Project Size:** (Grandezza del Progetto) Un Progetto più è grande, è più si necessita di comunicazione all'interno del team di sviluppo, se il tempo per lo sviluppo è ridotto si riduce di conseguenza la produttività individuale

**Technology Support:** (Tecnologie supportate) Buona tecnologie di supporto come i tools CASE, possono migliorare la produttività.

**Working Environment:** (Ambiente di Lavoro) Un Buon ambiente di lavoro con aree di lavoro private contribuisce a migliorare la produttività.

### **ESTIMATION TECHNIQUES** (Tecniche di stima)

Esistono tecniche di stima per determinare il prezzo finale di un software.

**-Algorithmic Cost Modelling** (Costo di modellazione algoritmica) : Modello Basato su informazioni dal costo storico che si riferisce a software già rilasciati usando una metrica (solitamente le dimensioni) per il costo finale del progetto. Il costo è stimato da una funzione matematica.  $Sforzo = A \times Size \times B \times M$ , dove A è una costante, B riflette lo sforzo sproporzionato e M è un moltiplicatore, il più usato per la stima dei costi è la dimensione del codice.

**-Expert Judgement** (Giudizio di Esperti) Vengono contattati esperti dello sviluppo Software, per avere una stima del costo del software che viene effettuata in base a una metrica e dallo sforzo che predice il modello.

**-Estimation by analogy** (Stima per Analogia ) si effettua una stima sulla base di progetti già esistenti inerenti al dominio applicativo che abbiamo sviluppato.

**-Parkinson Law** (Legge di Parkinson) La Legge di Parkinson afferma che il lavoro si espande per riempire il tempo a disposizione. Il costo è determinato dalle risorse disponibili anziché dagli obiettivi di valutazione. Se il software deve essere consegnato in 12 mesi e 5 persone sono disponibili, lo sforzo richiesto è stimato in 60 persone al mese.

**- Pricing to win** (Prezzo per vincere) Il costo del software è stimato in base ciò che il cliente ha a disposizione da spendere per il progetto. Lo sforzo stimato dipende dal budget del cliente e non sulla funzionalità del software.

## PRICING TO WIN

IL costo del progetto è determinato da ciò che può spendere il cliente

Vantaggi: È possibile ottenere il contratto.

Svantaggi: La probabilità che il cliente ottiene il sistema è piccola.

Questo approccio può sembrare immorale e in maniera poco professionale. Tuttavia, quando vengono a mancare informazioni dettagliate può essere l'unica strategia appropriata. Il costo del progetto è concordato sulla base di una bozza di proposta e lo sviluppo è vincolato da tale costo.

## TOP-DOWN AND BOTTOM UP ESTIMATION

Top-down

Si Avvia a livello di sistema e si valutano le complessive funzionalità e come questo viene consegnato attraverso sotto-sistemi.

Bottom-up

Avviare a livello di componente e valutare lo sforzo richiesto per ciascun componente. Si sommeranno questi sforzi per raggiungere una stima finale.

## COCOMO2

COCOMO 2 incorpora una serie di sotto-modelli che producono stime di software sempre più dettagliate.

I sotto-modelli in COCOMO 2 sono:

- **Applicazione del modello di composizione.** Utilizzato quando il software è composto da parti esistenti.

- **Modello di progettazione precoce.** Usato quando i requisiti sono disponibili ma progettazione non è ancora iniziata.

- **Modello riutilizzo.** Utilizzato per calcolare la ricerca di integrazione di componenti riutilizzabili.

- **Post-modello di architettura.** Usato una volta che il sistema di architettura è stata progettato e ulteriori informazioni sul sistema sono disponibili.

## MANAGING GROUPS (Gestione del Personale)

Il personale è l'organizzazione più importante all'interno di un assetto organizzativo per lo sviluppo di un progetto, il Manager dovrà orientarsi soprattutto a capire il personale su cui fare affidamento, se ci sarà incomprensione col personale la gestione è destinata a fallire.

Una cattiva gestione del personale è una delle cause principali per il fallimento del progetto di sviluppo.

## SELECTING STAFF (SELEZIONE DELLO STAFF)

Un compito importante all'interno della gestione di un Progetto è la selezione del personale.

Prima di selezionare un candidato bisogna raccogliere informazioni, effettuando numerose interviste oppure ottenere raccomandazioni o osservazioni da altre persone che hanno lavorato con il candidato in esame.



## MOTIVATING PEOPLE (Motivazione del Personale)

Un ruolo importante del manager che ha il compito di gestire il progetto è quello di motivare il personale che sta lavorando sul progetto.

Motivare non è facile in quanto il personale necessita di motivazioni differenti, sia a livello individuale che a livello di gruppo.

Inoltre quando si cerca di motivare si deve tener conto della personalità con cui si ha a che fare, Le personalità si suddividono in:

Task-Oriented: La motivazione di questo tipo di personalità è il lavoro stesso.

Self-Oriented: Il lavoro è un mezzo per ottenere soddisfazioni e raggiungere obiettivi personali

Interaction-Oriented: Lavorano perché vogliono stare a contatto con altre persone.

## GROUP COMPOSITION (Composizione di un gruppo di lavoro)

Come ribadito il manager ha il compito di creare il gruppo di lavoro, ma è molto importante capire le personalità da inserire, se un gruppo dovesse essere composto soltanto da persone con personalità:

Task-Oriented: Tutti farebbero ciò che vogliono

Self-Oriented: Tutti vorrebbero comandare

Interaction-Oriented: Ci sarebbero troppe chiacchiere e poco lavoro.

Quindi un gruppo efficace sarà composto da un po' tutte queste personalità.

Per la progettazione di piccoli software, un gruppo è organizzato in modo informale.

In particolare i gruppi informali, agiscono nel loro insieme e prendono decisioni a livello di gruppo, il capogruppo non è autorizzato a ripartire compiti, e il lavoro verrà suddiviso dal gruppo stesso e i compiti assegnati in base all'esperienza e alle capacità di chi ve ne fa parte.

Mentre per la progettazione di grandi sistemi software un gruppo potrà essere organizzato anche in gerarchie, in cui il gruppo generale è suddiviso in sottogruppi e vi sono diversi responsabili di questi sottogruppi.

Questo è un approccio che dà ottimi risultati se tutti i componenti del gruppo sono esperti e competenti

## GROUP LEADERSHIP (Leader del gruppo)

La leadership all'interno di un gruppo dipende dal rispetto non dalla posizione che si occupa.

Il leader potrebbe essere sia un tecnico che un dirigente amministrativo, la leadership democratica è più efficace rispetto a quella autocratica.

# SOFTWARE DEVELOPMENT

Lo sviluppo del software può essere categorizzato secondo la metodologia perseguita , e cioè :

- Rapid software development ( RAD)
  - RAD tools
  - Riutilizzo di sistemi standalone ( es. Database)
  - Sviluppo Agile
- Riutilizzo di altro software
- Sviluppo orientato al riutilizzo dei componenti ( Component-based )

## Rapid Software Development

Nell'ipotesi in cui non fosse possibile produrre un insieme di requisiti consistenti, causa repentini cambiamenti, il modello a cascata ( waterfall model ) risulterà impraticabile.

Per questo che si ricorre ad un processo di sviluppo iterativo e concorrente che si traduce in una serie di incrementi delle funzionalità del software di volta in volta testate e validate.

Lo sviluppo incrementale porta alcuni vantaggi : sviluppo prioritario in relazione alle direttive del cliente e quindi anche molto veloce, vista la continua partecipazione del cliente il sistema lo soddisferà meglio.

Gli svantaggi sono relativi soprattutto alla mancanza di un documento di specifiche senza il quale è molto difficile dimostrare i progressi, bisognerà adottare un'altra tipologia di contratto, risulterà complicato produrre dei test ai quali sottoporre il sistema, infine continui cambiamenti tendono a corrompere la struttura del software rendendo più costoso il raggiungimento di nuove specifiche.

Lo sviluppo incrementale e la prototipazione sono due tipologie in contrasto in quanto il primo sviluppa un sistema finito partendo dalle specifiche più chiare, il secondo deriva le specifiche sviluppando prototipi partendo dalle specifiche meno chiare.

L'ambiente RAD è composto da : Linguaggio per DB, generatore di interfacce, applicazioni office, e generatore di report, il tutto collegato ad un DBMS.

L'utilizzo di software standalone, come i database, è un ottimo modo per soddisfare delle specifiche per lo sviluppo RAD.

RAD si traduce in sviluppo Agile le cui caratteristiche in sintesi sono :

1. Il cliente deve partecipare attivamente allo sviluppo in quanto deve pensare,ordinare e valutare le specifiche del sistema.
2. Il software si compone di tanti incrementi di funzionalità
3. Non devono esistere dei processi di sviluppo, il team deve scovare il proprio workflow

4. Il sistema deve accomodare nel migliore dei modi i cambiamenti
5. Bisogna mantenere la semplicità

Lo sviluppo Agile però potrebbe soffrire dei seguenti problemi :

1. Gli sviluppatori potrebbero non riuscire a sostenere il ritmo intenso di sviluppo che caratterizza I metodi Agili.
2. Priorizzare è difficile, soprattutto nel caso di più stakeholder
3. Mantenere la semplicità richiede altro lavoro.
4. Il contratto rimane un problema.

Il più famoso e utilizzato dei metodi agili è l'**EXTREME PROGRAMMING**

Nella pratica l'extreme programming comporta:

1. Le specifiche sono memorizzare sotto forma di Story Card e Stories, le quali sono suddivise poi in TASKS dagli sviluppatori.
2. Lo sviluppo di funzionalità, anche piccole, è molto frequente ( si parla di più rilasci per giorno).
3. Si progetta il minimo indispensabile per soddisfare i requisiti
4. Prima di implementare nuove funzionalità si sviluppano un serie di test automatizzati per la verifica
5. **Refactoring, Refactoring, Refactoring**
6. Gli sviluppatori lavorano in coppie.
7. Non devono esistere parti di software la cui conoscenza del funzionamento è a capo di una sola persona.
8. Integrazione Continua con il sistema e continua esecuzione dei test
9. Il cliente deve essere sempre a disposizione del teamù

## Riuso del software

Alcuni sistemi sono progettati in modo da riutilizzare altri componenti di altri software, questo approccio dovrebbe permettere di sviluppare in maniera più efficiente , veloce e meno costosa.

Il riuso del software è possibile a vari livelli : semplici funzioni, riuso dei componenti, riuso di interi sistemi., design patterns e concetti ,frameworks(set di classi estendibili per lo sviluppo), COTS, Applicazioni configurabili composte da funzionalità generiche, ERP ( gestione ordini, e altri processi comuni di business)

*Un Design Pattern non è altro che la descrizione di un problema e la sua soluzione, nel modo più astratto possibile.*

## **Sviluppo basato sui componenti ( Component-based)**

Con questa metodologia si vuole superare l'incapacità di riuso in uno sviluppo OO, astruendo il più possibile i componenti.

Indipendenza , Standards così da facilitare l'integrazione, Interporabilità dei componenti stessi ed un processo di sviluppo basato principalmente sul riuso, sono le caratteristiche essenziali di questa metodologia.

I Componenti sono indipendenti dal linguaggio, e il loro sviluppo segue un modello di standard ed interfacce ben definito, come ad esempio il Corba.

## **SOFTWARE EVOLUTION**

Il software è inevitabilmente soggetto a cambiamenti in quanto : durante l'utilizzo posso emergere nuovi requisiti, cambia l'ambiente circostante in termini di business, la riparazione di errori posso evolvere in cambiamenti, l'equipaggiamento su cui gira il software cambia o il software è diventato troppo lento.

La gestione dei cambiamenti è un problema chiave per le organizzazioni.

Lo studio sui processi di cambiamento di un sistema è il Program evolution dynamics, il quale si compone di alcune leggi ( Lehman's laws)

Le leggi proposte da Lehman e Belady sono:

- Un programma è necessariamente soggetto a cambiamenti
- Evolvendosi accresce la sua complessità
- Grandezza, tempi di rilasci ed altri attributi generalmente restano invariati
- Lo tasso di sviluppo è solitamente indipendente dalla risorse a disposizione dal sistema.
- La Famigliarità del programma resta invariata
- Le funzionalità devono aumentare per mantare l'utente soddisfatto
- La qualità tenderà a peggiorare.
- Per ottenere migliorie significative è necessario lavoare con un sistema di feedback.

La manutenzione solitamente non modifica l'architettura del sistema e l'aggiunta o la modifica di

funzionalità è generalmente più costoso in termini di sforzi anziché la correzione degli errori o l'adattamento del programma.

Il costo di manutenzione si può tradurre in : stabilità del team ( meno cambiamenti = meno costi ), responsabilità contrattuale (niente responsabilità per la manutenzione = niente costi), abilità del team, struttura del programma.

Un sistema può essere re-ingegnerizzato cioè, si possono ristrutturare o ridocumentare interi sistemi o parti di essere senza cambiarne le funzionalità riducendo costi e rischi.

Far evolvere un sistema Legacy è una questione di scelte da parte dell'organizzazione proprietaria in quanto dipende tutto dalla qualità del sistema e dal suo valore monetario :

|               |              |                                |
|---------------|--------------|--------------------------------|
| Bassa Qualità | Basso Valore | Da buttare                     |
| Bassa Qualità | Alta Valore  | Sostituito o re-ingegnerizzato |
| Alta Qualità  | Basso Valore | Sostituito con COTS , o altro  |
| Alta Qualità  | Alta Valore  | Normale manutenzione           |

# TESTING & CONFIGURAZIONE

## TESTING

Alla penultima fase dello sviluppo di un prodotto software si colloca la fase di testing e configurazione del prodotto stesso.

In particolare la fase di testing è articolata in due sotto-fasi:

1. **Verifica**

si occupa di determinare se il software prodotto è conforme alle specifiche contrattuali.

2. **Validazione**

ha il compito di determinare se il prodotto creato fa veramente ciò che il cliente si aspetta.

La verifica e validazione oltre a far parte del processo di realizzazione di un software, creano a loro volta un processo che prende il nome di **V & V Process**. (Processo di Verifica e Validazione)

Il V&V Process può essere applicato a qualunque processo di sviluppo software, esso inoltre ha due principali obbiett:

- a) La scoperta di difetti implementativi e funzionali
- b) L'affidabilità del software in una situazione di reale utilizzo

Si può dedurre che attraverso il V&V PROCESS il cliente acquisirà nei confronti del software un certo livello di fiducia, che dipende dal fine di destinazione del software, dagli utenti che lo utilizzeranno e dall'ambiente di marketing:

a) **Il fine di destinazione**

Il livello di fiducia dipende dal tipo di aziende e/o organizzazione che lo utilizzerà  
(es. software per ospedale ha bisogno di alto livello di fiducia)

b) **Aspettative degli utenti**

Gli utenti potrebbero non essere molto fiduciosi del prodotto software  
(es. software home banking appena presentato, nessuno si fidava per paura di furti sul proprio conto)

c) **Ambiente di marketing**

In situazione di concorrenza spietata occorre presentare sul mercato un prodotto nuovo che non ha superato a pieno il processo V&V, poiché l'importante è che sia una novità.  
(es. Samsung e Apple)

### Tipologie di verifica software

Nella pratica reale la fase di verifica segue due metodologie differenti e complementari:

- A. **Ispezione del software o Verifica Statica**, si concentra all'analisi dettagliata del codice sorgente, che può avvenire anche con tools specifici.
- B. **Testing del software o Verifica Dinamica**, si osserva sulla base di determinati input i relativi comportamenti in output.  
Si adoperano due tipologie di testing:
  - 1) *Test dei difetti*, che si concentrano sulla scoperta di difetti nel sistema
  - 2) *Test di validazione*, hanno il compito di verificare se i requisiti sono stati soddisfatti o meno

## Testing & Debugging

I test dei difetti e il debugging sono due processi distinti, in quanto il test dei difetti ha compito di verificare l'esistenza di difetti presenti nel sistema, mentre il processo di debugging ha compito localizzare e correggere i difetti derivati dalla fase di testing.

### Piano di ispezione

Il piano di ispezione è un insieme di documenti che formalizza i test eseguiti sul prodotto con i relativi risultati. Inoltre in tali documenti vanno specificati eventuali rilevamenti dei difetti, senza specificare come sono stati corretti.

I difetti in un prodotto software, possono essere:

- 1) Errori logici
- 2) Scelte implementative in contrasto con l'architettura sulla quale sarà usato il prodotto
- 3) Anomalie nel codice che possono portare problemi alla compilazione(*variabile non inizializzata*)

### Fasi del processo di testing di un Software

1. Si progettano i singoli test Case con i rispettivi output
2. Si implementano i test Case progettati in precedenza e si crea un insieme omogeneo di test, che prende il nome di data Test
3. Si avvia il software con input più dataSet
4. Si compara l'output ottenuto dal programma con gli output effettivi del punto 1
5. Viene generato un report con i risultati

### Alcune regole di un testing efficace

1. Tutte le funzioni fornite da un interfaccia grafica hanno una maggiore probabilità di difetti, quindi devono essere testati con molto rigore.
2. Le combinazioni di azioni che conducono a una azione sull'applicazione hanno la stessa priorità di quelle dell'interfaccia grafica.
3. L'interazione tra utente e software deve essere testato con corretto e incorretto input.
4. In un approccio di tipo Object-Oriented occorre prestare particolare attenzione alle gerarchie della classi, e verificare "BENE" l'overriding.

**N.B.**

**Non esiste un processo di testing esaustivo in grado di affermare che un software è privo di difetti**

### Stress Testing

Il testing più usato è lo stress test, che ha il compito di sovraccaricare il sistema con grandi quantità di dati in input, così da determinare oltre alla correttezza dell'output anche la stabilità del sistema. Lo stress test è molto usato per software che interagiscono con molti client in rete, e per software che riguardano la sicurezza informatica

# CONFIGURAZIONE

La fase di configurazione si occupa delle problematiche legate alle diverse versioni del software, tali versioni dipendono da specifiche differenti, alcune di esse sono:

1. Diverso hardware o diverso SO
2. Offerte di funzionalità differenti
3. Particolari richieste del clienti

Uno schema di configurazione di un prodotto è quello relativo ai sistemi operativi, in quanto prima viene rilasciata il prodotto, supponiamo Windows XP, da questo prodotto in un secondo momento verrà rilasciata la configurazione Desktop e la configurazione Server.

Una corretta gestione di configurazione è accompagnata da relativa documentazione, nella quale viene trascritta:

1. Specifiche inerenti alla configurazione prescelta
2. Schema di progettazione
3. Software per la configurazione corrispondente
4. Data Test eseguiti
5. Manuale Utente finale

## Concetti Chiave

**Versione** è un'istanza di un sistema, ma con funzionalità distinte dal sistema originale

**Variante** è un sistema di identico a un altro, con le leggere differenze di funzionalità

**Release** un'istanza di un sistema che viene distribuito agli utenti finali, ovvero quel prodotto che termina il proprio processo di sviluppo.



# Patter Creazionali

I Patter Creazionali astraggono il processo di creazione degli oggetti rendendo il sistema sia indipendente dal modo in cui essi vengono creati.

Per archiviare questo obiettivo i pattern creazionali sfruttano l'ereditarietà della programmazione OOP, delegando l'istanziazione ad un'altro oggetto.

I Patter Creazionali sono:

- Factory Method – Metodo Fabbrica
- Abstract Metody – Fabbrica Astratta
- Singleton
- Prototype
- Builder

## Factory Method

L'intento è quello di creare un'interfaccia che si occupi della creazione degli oggetti, delegando il tipo concreto alle sue sotto classi.

Lo si può applicare quando si vuole delegare la creazione degli oggetti a delle sotto classi.

In questo pattern è necessario creare una sottoclasse della fabbrica per ogni tipo di oggetto concreto con il risultato di esplodere la gerarchia delle classi , e a meno che non si avesse già necessità di farlo è uno svantaggio.

FM si può implementare in due modi : parametrizzando i metodi fabbrica così da consentire una fabbrica multi prodotto, oppure semplicemente istanziando la sottoclasse di fabbrica relativa al tipo di oggetto che si vuole creare.

Altri pattern legati a questo sono l'Abstract Factory e il Prototype.

## Abstract Factory

L'intento è quello di creare un'interfaccia che permette la creazione di diverse famiglie di oggetti.

Lo si applica quando il sistema deve essere indipendente dalla creazione, composizione e rappresentazione degli oggetti, oppure quando un sistema deve creare oggetti di diverse tipologie, o ancora, quando si vuole nascondere l'implementazione degli oggetti esponendo un'interfaccia.

AF offre diversi vantaggi : isola le classi concrete, consente di mischiare le tipologie di oggetti facilmente semplicemente cambiando la fabbrica concreta, garantisce consistenza tra oggetti di famiglie diverse.

Piccolo svantaggio risiede nella difficoltà di creazione di nuove tipologie di oggetti, in quanto l'insieme dei prodotti che posso essere creati è fissato all'interno della classe.

Esistono diversi modi di implementare questo pattern: si può fare applicando il singleton alla fabbrica, si crea un'interfaccia applicando il metodo fabbrica o il prototype delegando a delle sotto classi concrete la creazione degli oggetti, oppure aggiungendo dei parametri per rendere flessibile la creazione.

Come già menzionato, l'Abstract factory si sposa perfettamente con il pattern FM, il Singleton e il Prototype

## Singleton

Il Singleton ci assicura che venga creata una sola istanza della classe e che venga fornito un sola interfaccia d'accesso globale.

Lo si applica quando si vuole esattamente una istanza della classe relegando alla classe stessa il compito di assicurarla, si vuole accedere all'istanza attraverso una e una sola interfaccia.

Conseguenze di questo pattern sono: il controllo dell'accesso all'istanza, la riduzione del name space-pollution, la possibilità di estendere la classe singleton e l'utilizzo a run-time della classe, il controllo del numero delle istanze.

Questo pattern si implementa nascondendo le operazioni di creazione in una funzione della classe, utilizzando un registro di classi singleton nel caso di una gerarchia di singleton.

Abstract Factory, Builder e Prototype sono dei pattern collegati al Singleton.

## Prototype

Questo pattern vuole creare un'interfaccia che permetta di specificare i tipi di oggetti da creare attraverso istanza di prototipi e che permetta di creare gli oggetti clonando quei prototipi.

Il pattern è applicato quando si vuole decidere il tipo a run-time l'oggetto da creare, quando si vuole ridurre la gerarchia del Abstract Method e ovviamente quando si vuole rendere indipendente il sistema da come gli oggetti sono creati, composti e rappresentati.

Vantaggi sono: si nasconde l'implementazione degli oggetti, si possono aggiungere/togliere oggetti al run-time, permette di comporre gli oggetti senza bisogno di programmare, riduce il numero di classi nelle gerarchie necessarie ad esempio nel FM.

Lo si implementa con un gestore di prototipi quando il numero dei prototipi non è definito a tempo di compilazione, implementando il metodo di Clonazione, e quando si vuole inizializzare i cloni implementando un metodo di Init.

Pattern collegati con Prototype sono l'AF, il Composite e il Decorator.

## Builder

L'intento è quello di separare la costruzione di oggetti ( complessi ) dalla loro rappresentazione, permettendo così diverse rappresentazioni utilizzando lo stesso processo di costruzione.

Si applica quando la creazione di un oggetti deve essere indipendente dai pezzi per la creazione e da come sono assemblati, quando il processo di costruzione deve permettere più rappresentazioni dell'oggetto che si costruirà.

Le conseguenze di questo pattern sono: un'interfaccia che nasconde come un'oggetto e rappresentato e come esso viene assemblato, una classe Builder che fornisce le direttive di costruzione dell'oggetto, l'isolamento del processo di costruzione dalla rappresentazione, infine consente un controllo preciso sul metodo di costruzione.

Builder si implementa con una classe astratta che definisce i metodi per ogni componente che il Director può richiedere, classi concrete che fanno override dei metodi;

Al Builder sono correlati l'Abstract Factory e il Composite.

## ***ADAPTER***

L'intento dell'Adapter è quello di convertire l'interfaccia di una classe nell'interfaccia che si aspetta l'utente. Permette, quindi, alle varie classi di lavorare insieme anche se inerenti a interfacce non compatibili fra loro.

L'Adapter viene usato principalmente quando una classe, progettata per il riuso, non può essere riutilizzata a causa della sua interfaccia.

Questo tipo di pattern viene usato quando:

- Si vuole usare una classe esistente ma la sua interfaccia non coincide con quella che serve all'utente
- Si vuole creare una classe riutilizzabile che fa collaborare altre classi, anche con interfacce non compatibili fra loro
- Si ha bisogno di utilizzare diverse sottoclassi esistenti ma l'adattamento dell'interfaccia di ognuna di esse non è praticabile. (SOLO ADATTATORE A OGGETTO)

I client chiameranno solamente le operazioni sull'istanza dell'Adattatore, il quale si farà carico di processare tali richieste tramite chiamate di funzioni dell'Adattato (Interfaccia non compatibile).

### **CONSEGUENZE**

#### ***ADATTATORE DI CLASSE***

- 1) Adatta l'ADATTATO alla classe che ne ha bisogno (TARGET) tramite la creazione di una concreta classe Adapter.
- 2) Un adattatore di classe non funzionerà se si vuole adattatore una classe e tutte le sue sottoclassi.
- 3) Lasciare che l'Adattatore sovrascriva alcuni metodi dell'Adattato.
- 4) Introduzione di un solo oggetto: nessun bisogno di puntatori addizionali per raggiungere l'Adattato.

#### ***ADATTATORE A OGGETTO***

- 1) Lascia che un singolo Adattatore lavori con più Adattati (l'adattato e le sue sottoclassi)
- 2) L'Adattatore può aggiungere funzionalità agli Adattati.
- 3) Maggiore difficoltà nel sovrascrivere i metodi dell'Adattato (è necessario che l'adattatore si riferisca alla sottoclasse specifica piuttosto che all'Adattato generico).

L'Adattatore, quindi, può svolgere diversi compiti di adattamento, dalla semplice ridenominazione dei metodi all'aggiunta di nuove funzionalità.

Un modo per implementare tale pattern in C++ sarebbe di far ereditare in modo pubblico il Target e in modo privato l'Adattato all'Adattatore.

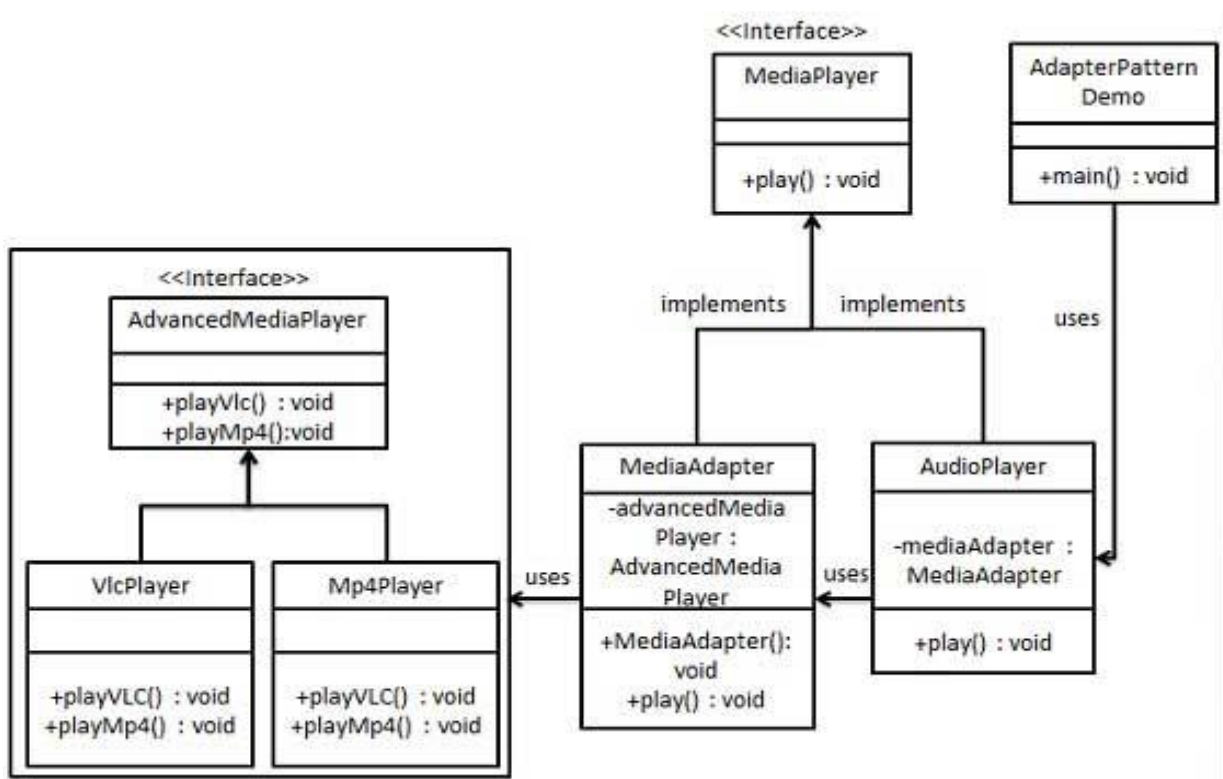
### Esempio:

Abbiamo un'interfaccia MediaPlayer e una classe concreta AudioPlayer che implementa l'interfaccia MediaPlayer e che può leggere solo formati mp3 di default.

Avremo, quindi, un'altra interfaccia AdvancedMediaPlayer e classi concrete che la implementano. Queste classi possono riprodurre formati mp4 e wmv.

Vogliamo che anche la classe AudioPlayer sia in grado di riprodurre questi formati.

Per ottenere ciò basta creare un adattatore MediaAdapter che implementa l'interfaccia MediaPlayer e che usa oggetti di AdvancedMediaPlayer per riprodurre gli altri formati. AudioPlayer userà quindi l'adattatore MediaAdapter che riprodurrà il tipo di formato richiesto, senza sapere dall'esterno però quale classe specifica lo riprodurrà realmente.



# Bridge (BR)( Ponte)

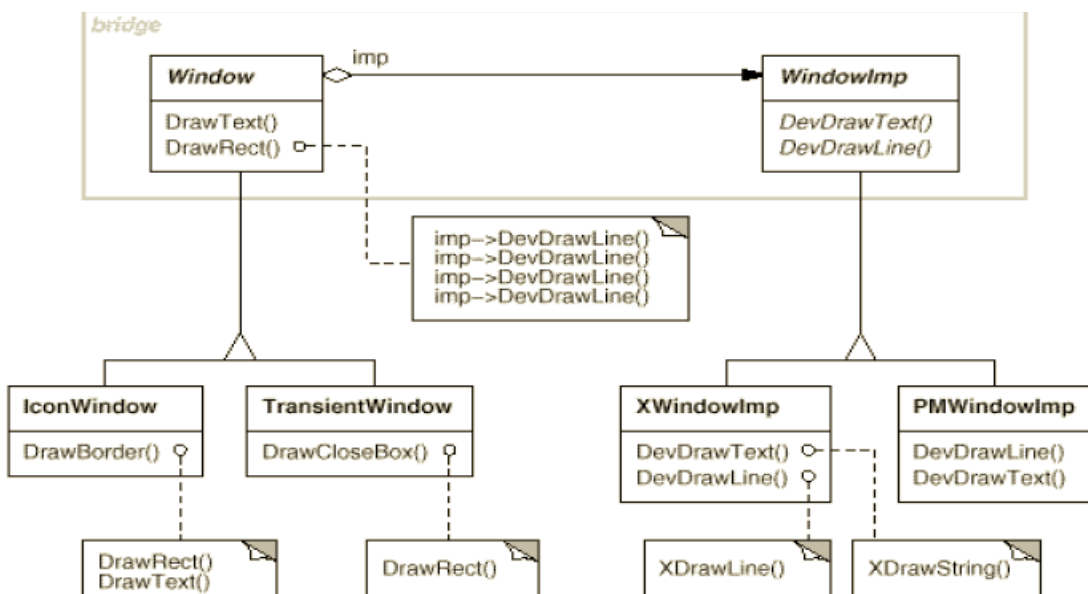
L'intento del bridge è quello di disaccoppiare un'astrazione reale dalla sua implementazione, cosicché astrazione e implementazione siano indipendenti, e possano variare l'una dall'altro.

Un'astrazione può avere diverse possibili implementazioni relative al suo dominio applicativo. Il metodo principalmente utilizzato è quello di usare l'ereditarietà sulle implementazioni.

Una classe astratta definirà l'interfaccia dell'astrazione in esame, e le sottoclassi concrete la implementeranno in diversi modi, ma questo non è un modo abbastanza flessibile in quanto l'ereditarietà sarà vincolata in modo permanente all'astrazione, il Bridge verrà utilizzato proprio per risolvere questa problematica.

Analizziamo un esempio:

Supponiamo di voler implementare l'astrazione di una FinestraUtente che appaia in diversi modi a seconda del SO del calcolare come per esempio XWindow e IBMManager, utilizzando il Pattern Bridge avremo:



Partecipanti nel Bridge(riferendoci all'esempio di sopra)

-Abstraction (Window)

Definisce l'interfaccia dell'astrazione, conterrà un riferimento ad un Implementor concreto (WindowImp) in modo tale che alla chiamata dei metodi, nell'astrazione ci riferiremo a quelli dell'implementor concreto.

-Ridefinizione dell'astrazione (IconWindow or TransientWindow)

Estendono l'interfaccia definita dall'astrazione.

-Implementor (Implementatore Concreto) (WindowImp)

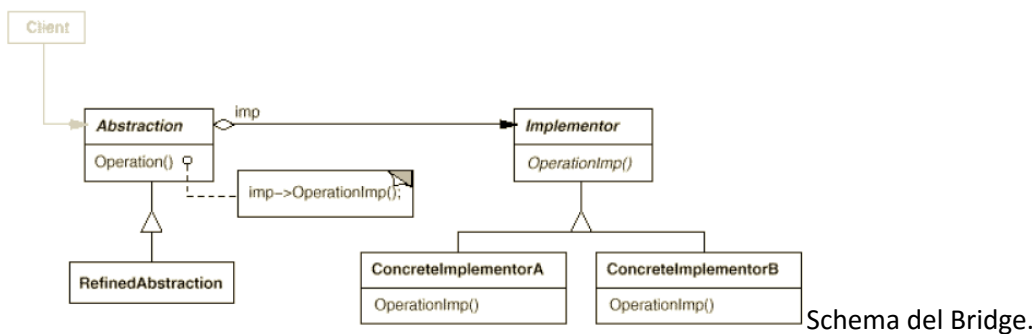
Definisce un'interfaccia per le implementazioni concrete, Implementor e Astrazione avranno interfacce diverse.

-Implementor Concreto (XWindowImp, PmWindowImp)

Implementa l'interfaccia dell'Implementor a definiranno un implementazione concreta, saranno utilizzate quando il riferimento dell'implementor all'interno dell'astrazione farà riferimento a una delle sottoclassi di implementazione concreta.

Utilizziamo il Bridge quando:

- Vogliamo evitare un legame permanente tra un'astrazione e la sua implementazione
- L'implementazione dev'essere selezionata o cambiata a run-time
- Sia le astrazioni che le implementazioni dovrebbero essere estensibili per le sottoclassi, il BR consente appunto di combinare le diverse astrazioni e implementazioni ed estenderli l'una dalle altre in modo indipendente
- I cambiamenti nell'implementazione non richiedono la ricompilazione dell'astrazione.
- (In c++) per nascondere l'implementazione di un astrazione al client



Bridge Conseguenze:

- Un implementazione non è legata in modo permanente ad un'altra interfaccia.
- Elimina a tempo di compilazione le dipendenze dall'implementazione
- Incoraggia la stratificazione che può portare ad una migliore strutturazione del sistema.
- Migliore Estensibilità: Sarà possibile estendere le Gerarchie di Astrazioni e le implementazioni in modo del tutto indipendente
- Sarà possibile nascondere i dettagli di implementazione al client

Potrà sussistere un caso in cui abbiamo un solo implementor.

- E' un caso degenere
- In situazioni come queste non è necessario creare una classe astratta per l'implementor, ma direttamente una classe concreta.
- La separazione dall'astrazione all'implementor è ancora utile, in quando i cambiamenti futuri nell'implementor non dovrebbero richiedere la ricompilazione dell'astrazione.

Pattern Correlati: Abstract Factory , Adapter.

# COMPOSITE (CO) (COMPOSITO)

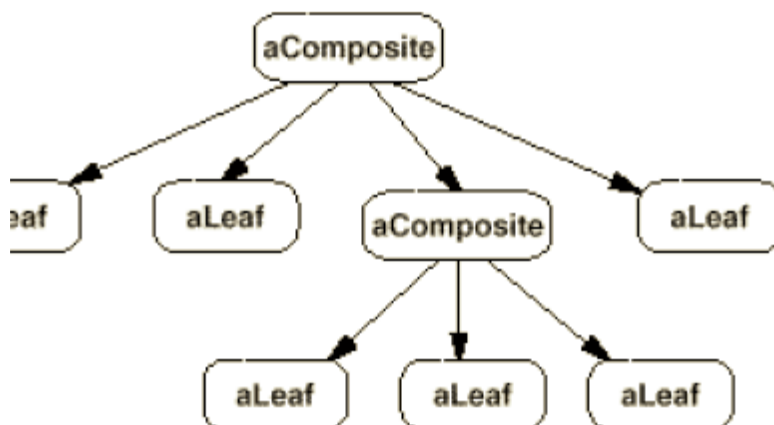
Il suo intento è quello di rappresentare gerarchie di oggetti, con una struttura ad albero.

Il Composite consente ai Client di trattare i singoli oggetti e le composizioni in modo uniforme.

Il pattern Composite potrà essere una classe astratta che rappresenterà sia le primitive e sia i suoi oggetti composti, attraverso operazioni di accesso e gestione dei figli.

E' utile utilizzare il Composite quando:

- Si vogliono rappresentare tutte le parti della gerarchia degli oggetti in esame.
- Vogliamo che il Client siano in grado di ignorare la differenza tra le composizioni di oggetti e gli oggetti individuali.
- I Client tratteranno tutti gli oggetti della struttura composta in modo uniforme.



Il composite come si vede dalla figura, avrà un'interfaccia comune a tutti, e ogni sottoclasse potrà essere:

-Foglia, ovvero l'oggetto che non potrà aggiungere al suo interno nient'altro in quanto rappresenta l'unità atomica della nostra struttura.

-Figlio, ovvero l'oggetto che al suo interno potrà esser composto da foglie, e potrà gestire e accedere ai suoi figli.

Bisognerà:

- Dichiarare un'interfaccia per gli oggetti che faranno parte del Composite.
- Dare un'implementazione di default all'interfaccia comune a tutte le classi.
- Dichiarare un'interfaccia per gestire ed accedere alle componenti figlie.

I client utilizzeranno la classe Component per interagire con gli oggetti della struttura composta.

Se il destinatario sarà una foglia la richiesta potrà esser gestita direttamente, se invece il destinatario sarà un oggetto composto, solitamente esso inoltrerà le richieste al suo componente figlio.

Conseguenze dell'utilizzo del Composite:



Esso definisce quindi gerarchie di classi che consistono in oggetti primitivi (Foglie) o oggetti composti (Figli).

Il Client riuscirà ad utilizzare oggetti composti come se fossero primitive senza rendersene conto.

Sarà più facile aggiungere nuovi tipi di Componenti, in quanto i nuovi oggetti composti o le sottoclassi foglia lavoreranno sempre con le stesse strutture esistenti, e i client non dovranno apportare modifiche dopo l'inserimento di queste nuovi classi.

Tutto ciò potrà rendere la struttura del progetto molto generale, e questo potrà tendenzialmente essere un problema.

Implementazione del Composite:

- Riferimenti espliciti agli oggetti composti, semplificherà la traversata e la gestione della struttura composita stessa, sarà facile risalire lungo la struttura o eliminare un componente.

- Sarà essenziale Mantenere la coerenza della struttura, ovvero cambiare un parent Component soltanto quando è stato aggiunto o rimosso da un composito.

- Condivisione dei componenti, utile per:

  - Ridurre i requisiti di storage.

- Massimizzare l'interfaccia Component, per rendere i client ignari della foglia specifica o della classe composita che usano

  - La classe composita dovrebbe definire più operazioni comuni possibili.

  - La classe composita solitamente dovrebbe prevedere un implementazione di default per determinante operazioni, e le foglie e classi composite dovrebbero implementarle.

- La classe Composite (la radice della gerarchia) dovrà implementare le operazioni di add e remove.

Patterns Correlati

- Iterator

- Flyweight

- Decorator

- Visitor

# DECORATOR (DE) DECORATORE

Il suo intento è quello di aggiungere responsabilità aggiuntive ad un oggetto dinamicamente.

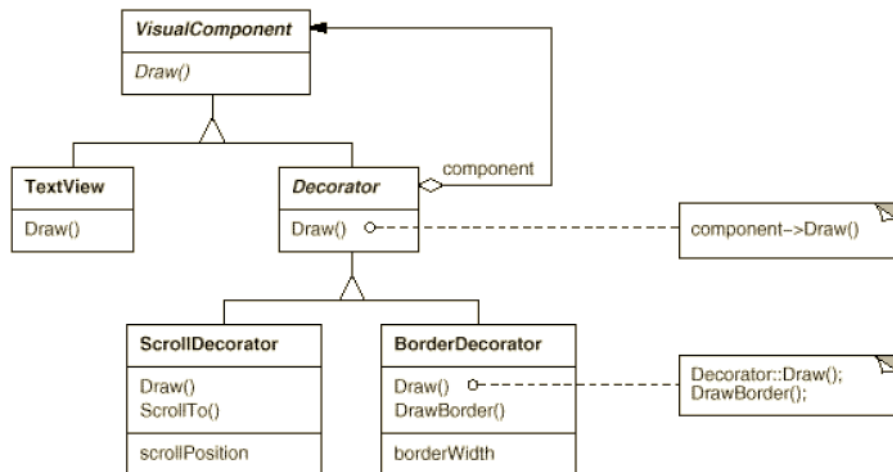
Dovremmo aggiungere responsabilità ai singoli oggetti, non a classi intere.

Un modo per aggiungere queste responsabilità, è usare l'ereditarietà.

Un approccio più flessibile è racchiudere un componente all'interno di un altro oggetto.

L'oggetto che racchiude altri oggetti, è detto decoratore

La trasparenza degli oggetti ci consente di realizzare decoratori in modo ricorsivo, dandoci la possibilità di aggiungere un numero illimitato di operazioni aggiuntive.



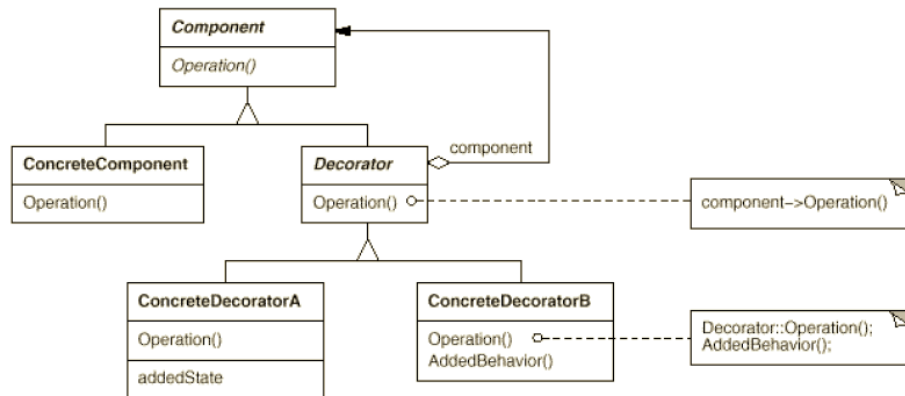
Come si vede data l'interfaccia **VisualComponent**, **Decorator** sarà una sottoclasse di essa, in quanto potrà essere una classe comune ad oggetti come le Scroll Bar o l'aggiunta di Bordi, infatti il decoratore ci consente di aggiungere operazioni aggiuntive diverse ad ognuno di essi indifferentemente facendo riferimento sempre a un metodo della classe padre.

I Client non dovranno poter dire quale sia la differenza tra un Componente Decorato e uno non Decorato.

Il Decoratore quindi viene usato per:

- Aggiungere responsabilità ad oggetti Individuali dinamicamente e in modo trasparente senza portare effetti sugli altri oggetti (vedi metodo `ScrollTo()` nell'immagine di sopra in **ScrollDecorator**)
- Quando l'estensione delle sottoclassi è impraticabile.

- Quando la definizione della classe è nascosta o altrimenti non è disponibile per la creazione di sottoclassi



E i partecipanti in questo Pattern saranno:

**Component:** definisce l'interfaccia per gli oggetti che potranno avere responsabilità aggizionali da dover aggiungere.

**ConcreteComponent:** definisce un implementazione concreta dell'interfaccia effettuando un operazione concreta.

**Decorator:** La classe decorator mantiene sempre un riferimento all'Interfaccia padre **Component**, e definirà un interfaccia conforme a quella **Component**, Decorato servirà appunto per creare le varie sottoclassi degli oggetti ad voler decorare.

**ConcreteDecoratorA:** implementerà in modo concreto **Decorator** e aggiungerà in riferimento all'interfaccia padre operazioni concrete al **Component** relative al suo caso d'uso.

Il **Decorator** inoltrerà le sue richieste ai suoi **Componenti** e le operazioni aggiuntive potranno essere inoltrate prima o dopo la richiesta.

Usare il Pattern Decorator offre alcuni Benefici:

- Maggiore flessibilità rispetto ad un ereditarietà statica
  - o Le operazioni aggiuntive sui componenti potranno essere aggiunte e rimosse a run time.
  - o L'ereditarietà ci consentirà di creare nuove classi per ogni responsabilità addizionale da dover aggiungere
- Evitare classi piene di funzionalità all'inizio della gerarchia
  - o Offre un approccio paga e vai per aggiungere nuove responsabilità

Inconvenienti nell'uso del Decoratore

- Decoratore e le sue componenti non sono identici
- Molti Oggetti Piccoli
  - o Il design si potrà tradurre spesso in sistemi composti da molti piccoli oggetti che sembrano tutti uguali tra di loro
  - o Gli oggetti si differenzieranno soltanto per il modo con cui sono interconnessi
  - o Facile da personalizzare ma difficile da imparare ed eseguire il debug.

Bisognerà Mantenere le classi dei Componenti in modo leggero

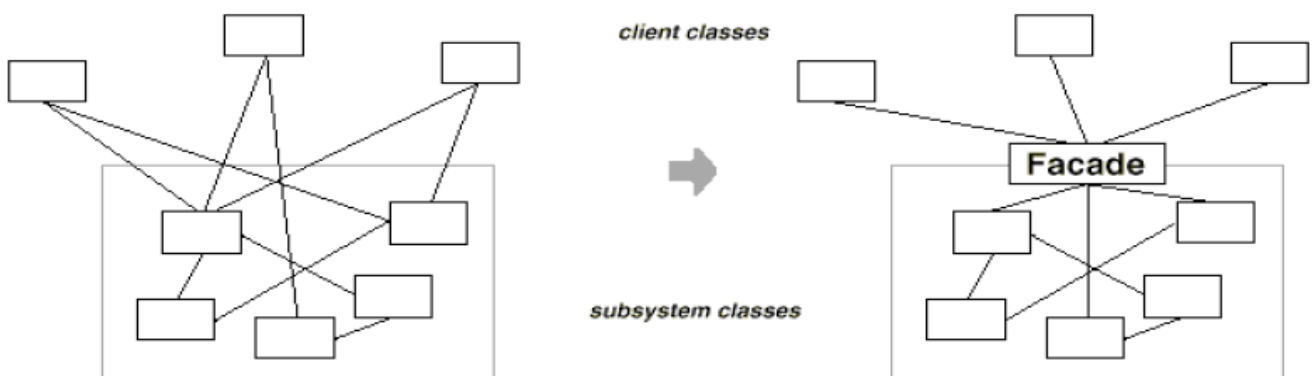
Componenti e decoratori dovranno derivare da un classe Component comune, questa classe dovrà essere inoltre leggera, altrimenti utilizzare decoratori molti pesanti ne inficerà la quantità, e sottoclassi concrete ne pagheranno per caratteristiche di cui non hanno bisogno.

Pattern Correlati:

- Adapter
- Composite
- Strategy

## Facade (Facciata) [FA]

- **Intento:** fornire un'interfaccia unificata di un insieme di interfacce di un sottosistema. La Facade definisce un'interfaccia di alto livello che facilita l'uso del sottosistema.
- **Motivazioni:** Strutturare un sistema in sottosistemi aiuta a ridurre la complessità. L'obiettivo comune di progettazione è MINIMIZZARE la comunicazione e le dipendenze tra i sottosistemi. Introdurre un oggetto Facade che fornisce in modo semplificato e unico l'interfaccia al sottosistema.



Consideriamo uno strumento di sviluppo dove le applicazioni possono accedere al sottosistema "Compilatore".

La maggior parte dei clienti che utilizzano il compilatore non si preoccupano dei dettagli di compilazione; in effetti i clienti vogliono solo compilare il codice. La **soluzione** a tale problema è offrire un'unica interfaccia ad alto livello che può proteggere i clienti da queste classi specifiche che compongono "Compiler", ovvero offrire l'interfaccia "Compiler" (FacadeCompiler).

- **Applicabilità:** usare il pattern Facade quando:
  - **Voler offrire un'interfaccia semplice per un complesso sottosistema.** Questo rende il sottosistema molto riutilizzabile e facile da personalizzare (in effetti il FA nasconde la fase di refactoring). Ma diventa anche più difficile da usare per i clienti che non hanno la necessità di personalizzare.
  - **Voler offrire una semplice visualizzazione di default del sottosistema che è abbastanza utile per i clienti.** Solo i clienti che hanno la necessità di personalizzare il sottosistema potranno guardare oltre la facciata, ovvero utilizzare le interfacce che compongono il sottosistema specifico.

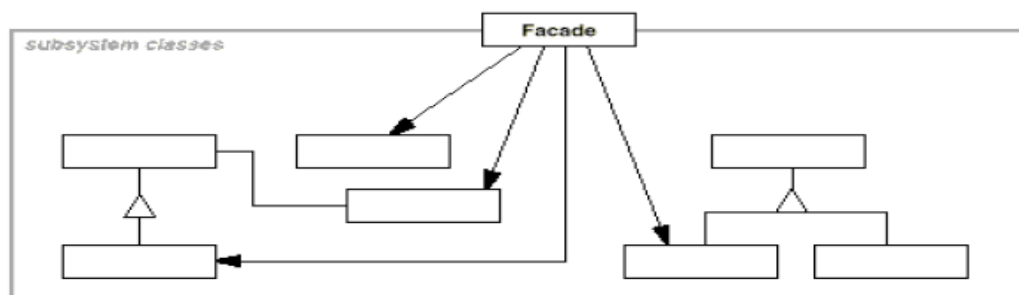
- **Ci sono molte dipendenze tra i clienti e l'implementazione delle classi** (nel caso della compilazione, senza l'interfaccia Compiler il cliente dovrebbe richiamare ogni interfaccia del sottosistema per compilare come linked, crea eseguibile, ecc...).

Il FA disaccoppia il sottosistema dai clienti e da altri sottosistemi, ovvero favorisce molti sottosistemi indipendenti e portabilità.

- **Voler sovrapporre i sottosistemi.**

Utilizzare l'FA per definire un punto di accesso per ogni livello di sottosistema. I sottosistemi comunicano attraverso le facciate; in questo modo si ha una semplificazione delle dipendenze tra i sottosistemi.

- **Struttura:**



- **Collaborazioni:** i clienti comunicano con il sottosistema attraverso la Facade, la quale è seguita da sottosistemi di oggetti. Gli oggetti del sottosistema svolgono l'attuale lavoro richiesto. La Facade traduce la propria interfaccia, in interfacce del sottosistema.

I clienti che usano la Facade non hanno l'accesso agli oggetti del sottosistema direttamente.

- **Conseguenze:**

- La Facade fa da scudo per i clienti dai componenti del sottosistema.
- Ridurre il numero di oggetti che il cliente manipola, rendendo il sistema più facile da utilizzare.
- Promuovere l'accoppiamento tra i clienti e il sottosistema, che consente di variare i componenti del sottosistema senza alterare il cliente.
- Aiuta a sovrapporre le dipendenze tra gli oggetti.
- Può eliminare dipendenze circolari o complesse; potrebbe essere importante quando il cliente e il sottosistema sono implementati indipendentemente.

- Ridurre le dipendenze di compilazione; minimizzare la ricompilazione quando il sotto sistema cambia; semplificazione di portabilità del sistema su altre piattaforme.
- Non impedisce le applicazioni dall'uso delle classi direttamente del sottosistema senza passare per la facciata, se questa operazione è volontaria. Quindi si valuta tra facilità di utilizzo e generalità.
- **Pattern Relativi:** Abstract Factory, Mediator, Singleton.

## FLYWEIGHT (PESO MOSCA) [FL]

- **Intento:** Usa per la condivisione di un grande numero di oggetti piccoli in modo efficiente.
- **Motivazione:** le strutture di oggetti complessi possono semplificare la progettazione di un'applicazione ma un'implementazione stupida può essere molto costosa (anche dal punto di vista di spazio, di memoria sprecata). Per esempio, si consideri un editor di documenti che ha tabelle e figure incorporate. La struttura dell'oggetto applicazione potrebbe mimare la struttura fisica del documento. Nel caso in cui si vuole scrivere una parola, saranno istanziate le lettere uguali più volte quindi causando uno spreco di memoria. Progettare in questo modo è buono, ma ci sono degli svantaggi come: potrebbe facilmente richiedere una quantità eccessiva di oggetti caratteri, consumo di molta memoria, e potrebbe incorrere in un accettabile sovraccarico a run-time!

Un flyweight è una condivisione di un oggetto che può essere usato in più "contesti" simultanei.

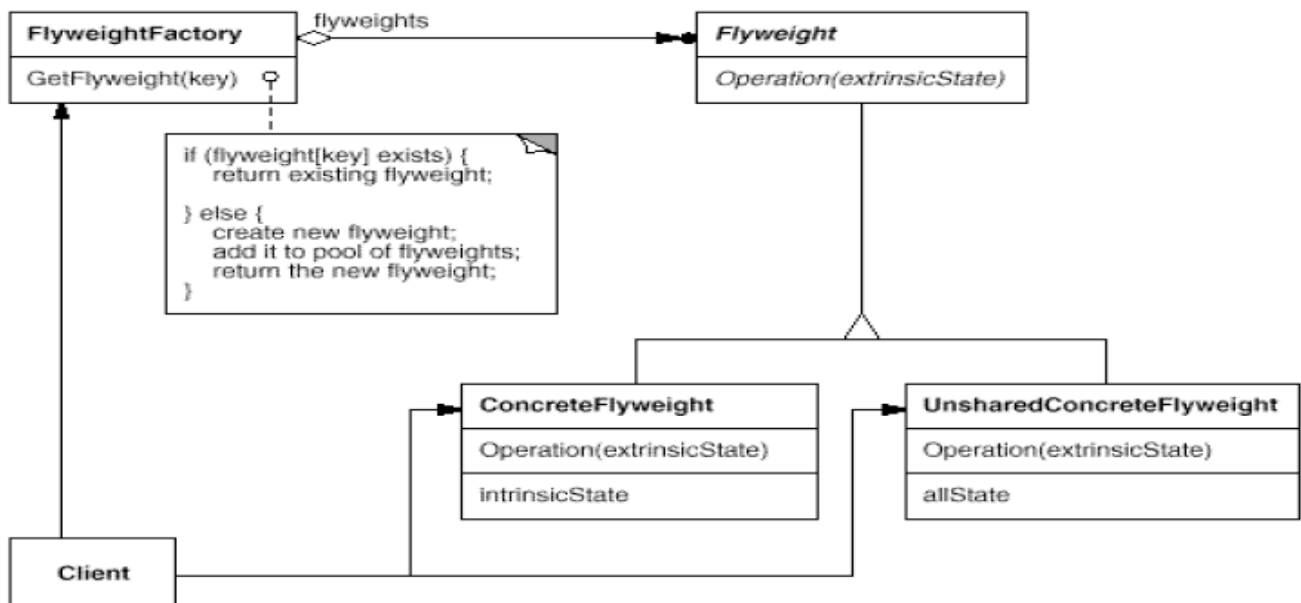
In particolare lo stato è definito in due modi:

- **Intrinseco:** contenuto/memorizzato nel flyweight; informazione che è indipendente dal contesto del flyweight.
  - **Estrinseco:** dipende dal contesto del flyweight; ovviamente non può essere condiviso per tutti i contesti; il client passa lo stato estrinseco al flyweight quando è necessario.
- **Applicabilità:** un flyweight deve possedere tutte le seguenti condizioni:
    - Un'applicazione usa un grande numero di oggetti.
    - I costi di memorizzazioni sono alti, poiché c'è una grande quantità di oggetti.
    - Lo stato di molti oggetti può essere costruito estrinsecamente.
    - Molti gruppi di oggetti possono essere replicati con pochi oggetti condivisi una volta che lo stato estrinseco è rimosso.



- L'applicazione non dipende dagli oggetti identici (si intende oggetti uguali ma concettualmente diversi). Ovviamente in questo caso bisogna stare attenti, poiché se l'applicazione dipende dagli oggetti identici allora gli oggetti FL che possono essere condivisi risulteranno identici se vengono testati ma concettualmente sono diversi tra loro.

- **Struttura:**



- **Collaborazioni:** un FL deve essere caratterizzato dall'avere entrambi gli stati intrinseco ed estrinseco.  
 Lo stato intrinseco è memorizzato dentro l'oggetto flyweightConcreto.  
 Lo stato estrinseco è memorizzato o elaborato dal Client.  
 I Clienti passano questo stato al flyweight quando devo effettuare delle operazioni su tali oggetti.
  - I clienti non dovrebbero istanziare direttamente la classe ConcreteFlyweight.
  - Infatti, i clienti devono istanziare gli oggetti ConcreteFlyweight esclusivamente dal FlyweightFactory.
- **Conseguenze:** Il FL potrebbe introdurre **costi** a run-time (**trasferire, cercare, ed elaborare lo stato estrinseco**). Tali costi sono compensati

dal risparmio di spazio. Tali costi aumentano più sono i flyweight da condividere.

**Fattori per il risparmio dello spazio:** la riduzione del numero totale di istanze come conseguenza della condivisione. La quantità di stato intrinseco per oggetto.

**Più flyweight sono condivisi, e più si ha un risparmio di spazio.**

**Ovviamente, lo spazio risparmiato incrementa con la quantità di stati condivisi.** Si può risparmiare spazio in due modi: (1) con la condivisione si riduce il costo dello stato intrinseco (2) e si ha uno scambio dello stato estrinseco a tempo di esecuzione.

**Il pattern FL è combinato spesso con il pattern Composito (per la rappresentazione di strutture albero).** Le nodi foglia flyweight non possono memorizzare un puntatore al proprio genitore. Il puntatore genitore è passato al flyweight come parte dello stato estrinseco.

- **Pattern Relativi:** Composite, State, Strategy.

# PROXY

L'intento del Proxy è quello di offrire un "surrogato" di un altro oggetto per controllare l'accesso a esso.

Il proxy, quindi, prende il posto dell'oggetto reale, assumendone lo stesso comportamento e si occupa della sua istanziazione quando viene richiesto.

Il pattern "Proxy" è applicabile ogni qual volta si sente il bisogno di una referenza più versatile e sofisticata di un semplice puntatore. Comuni situazioni di applicabilità sono:

- Proxy Remoto: offre una rappresentazione locale per un oggetto che si trova in un altro posto
- Proxy Virtuale: crea oggetti pesanti su richiesta così da ottimizzare l'applicazione
- Proxy Di Protezione: controlla l'accesso all'oggetto originale
- Referenza Smart: offre un rimpiazzo ad un semplice puntatore che è in grado di compiere azioni aggiuntive quando l'oggetto è acceduto.

Il proxy potrebbe riferirsi ad un Soggetto generico se tale Soggetto e il SoggettoReale hanno la stessa interfaccia.

Il proxy, poiché fa da surrogato all'oggetto reale, offre la stessa interfaccia a quella del Soggetto e può essere responsabile della sua creazione ed eliminazione.

Ulteriori responsabilità dipendono dal tipo del Proxy che viene usato:

- proxy remoti sono responsabili per codificare e inviare richieste al reale oggetto remoto
- proxy virtuali possono post-porre l'accesso e la creazione all'oggetto reale
- proxy di protezione, invece, controllano che colui che vuole effettuare una richiesta all'oggetto reale abbia i permessi richiesti per effettuare tale operazione.

Il pattern Proxy, in definitiva, introduce un riferimento indiretto quando si prova ad accedere all'Oggetto.

L'utilizzo di proxy, specialmente per il tipo virtuale, comporta la creazione dell'oggetto su richiesta. Copiare tale oggetto quando è soggetto solamente a operazioni di scrittura può ridurre di molto il carico dell'intero sistema e tale prezzo viene richiesto solamente quando l'oggetto deve essere realmente modificato.

Il proxy, inoltre, non deve essere a conoscenza del tipo specifico dell'oggetto reale poiché ne utilizza l'interfaccia astratta. Solamente nel caso in cui il Proxy dovrà istanziare un Oggetto Specifico dell'interfaccia, esso dovrà conoscere anche tale istanza specifica.

## CATENA DI RESPONSABILITA'

### Perché nasce la Catena Di Responsabilità?

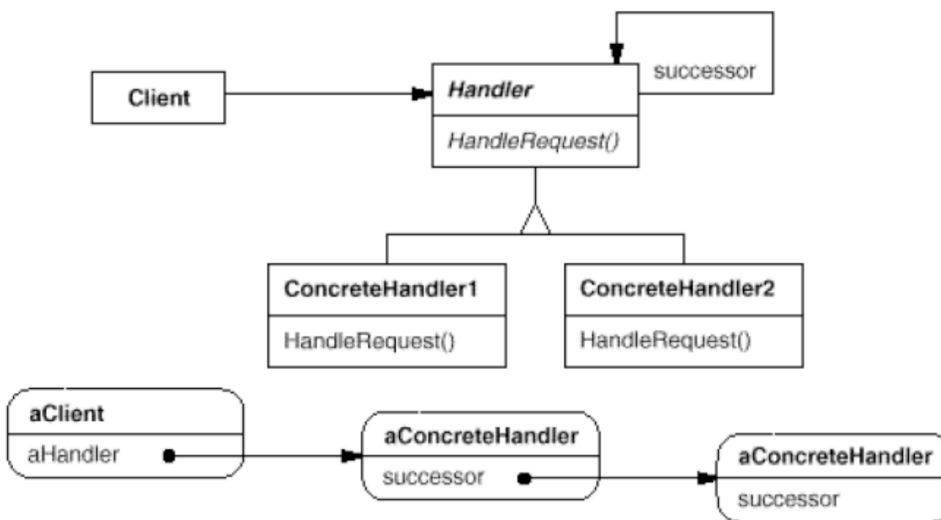
La catena di responsabilità è un pattern utilizzato per gestire la comunicazione tra molti oggetti. In particolare l'obiettivo di questo patter è di dissociare la comunicazione tra due oggetti, per molteplici problematiche, alcune di esse sono

1. La comunicazione potrebbe non essere diretta e quindi passare per più oggetti
2. Prevedere in ogni oggetto i collegamenti verso altri oggetti risulta poco efficiente

### Applicabilità

Molti oggetti devono risolvere richieste, senza sapere come l'oggetto in grado di risolvere la specifica richiesta la risolverà

### Com'è strutturato?



Il clienti richiede di risolvere una richiesta.

Questa sarà gestita dal gestore di richieste, il quale prova a risolverla con un oggetto a suo disposizione, se riesce bene, se l'oggetto non è in grado di risolvere la richiesta questa viene propagata ad altri oggetti, ripetendo questo procedimento fino a quando non si trova l'oggetto che riesce a risolvere quella specifica richiesta. Il propagare le richieste forma una catena, da qui il nome del pattern.

### Esempio

Supponiamo di voler implementare il comportamento del bottone help presente in quasi tutte le applicazioni. L'alternativa poco efficiente sarebbe di lasciare in ogni pannello la singola implementazione dello stato in cui l'applicazione si trova, questo però è poco efficiente in quanto posso visionare solo l'implementazione presente nello stato in cui mi trovo e non altre informazioni relativi ad altri oggetti.

A questo punto, utilizzando il pattern si ottiene una soluzione più efficiente, in quanto ci sarà un gestore delle richieste che sulla base delle richieste invocherà direttamente o indirettamente l'oggetto risolutore della richiesta

### Pattern Correlati

**Composite**, in quanto la struttura di propagazione delle richieste può essere intuitivamente dedotta come la composizione di più oggetti per la risoluzione di una richiesta.

## INTERPRETER

L'obiettivo del pattern interprete è quello di comprendere una frase di un determinato linguaggio data una grammatica. In particolare gli elementi fondamentali del pattern sono:

1. Linguaggio
2. Grammatica (insieme di regole che sanciscono come deve essere formata una frase)
3. Frase da interpretare

### Quando si usa...

L'interprete si usa quando si ha un linguaggio da comprendere, e le regole di grammatica in esso presente possono essere rappresentate con una struttura ad albero (esempio espressione aritmetica, prima si esegue la moltiplicazione, poi addizione o sottrazione).  
Notare che il pattern risulta molto efficiente quando le regole di grammatica sono semplici, in quanto con regole complesse la gestione dell'albero diventa ingestibile.

### Com'è strutturato ?

La struttura del pattern prevede un CONTESTO che sarà l'oggetto da interpretare, e inoltre l'interprete che si occuperà di comprendere il contesto.

### Conseguenze

- Cambiare la sintassi è molto facile se si ha una classe per rappresentare la grammatica
- Potrebbe essere molto complesso dal punto di vista computazionale gestire molte regole, in quanto ricordiamo che ogni regola è un nodo di un albero
- L'interpretazione di una nuova espressione è molto semplice, in quanto occorre semplicemente definire la grammatica con la quale è stata elaborata la nuova espressione.

### Pattern Correlati

1. Composite
2. Iterator
3. Flyweight
4. Visitor

# Mediator Pattern

## Intento:

L'intento è definire un oggetto che gestisca le interazioni tra più oggetti.

Il ME evita riferimenti espliciti tra oggetti, consentendo di variare l'interazione indipendentemente da essi.

## Motivazioni:

OOD incoraggia la distribuzione del comportamento in diversi oggetti ma, nel caso di molti oggetti e quindi di relazioni molti a molti, modificare il comportamento del sistema risulterà difficile.

Questo problema si risolve incapsulando il comportamento in un oggetto Mediator a parte, il quale:

- Controllerà e coordinerà l'interazione di gruppi di oggetti.
- Manterrà i riferimenti espliciti degli oggetti.
- Gli oggetti saranno a conoscenza del solo Mediator.

## Quando applicare questo pattern ?

Il Mediator si applica quando un gruppo di oggetti comunica in una maniera ben definita ma complessa, rendendo difficile distinguere le varie interdipendenze.

Riutilizzare un oggetto risulta difficoltoso perché comunica e fa riferimento ad molti altri oggetti.

Quando un comportamento distribuito tra più classi deve essere personalizzabile senza l'utilizzo di altre sotto-classi.

## Conseguenze:

Questo pattern limita le sotto-classi localizzando il comportamento delle interazioni tra oggetti, così che la sua modifica implica soltanto il coinvolgimento del Mediator e gli oggetti possono essere usati così come sono.

Disaccoppiando gli oggetti essi posso essere usati indipendentemente dal Mediator e vice versa.

Sostituisce interazioni molti-a-molti con una-a-molti, più facile da capire, mantenere ed estendere.

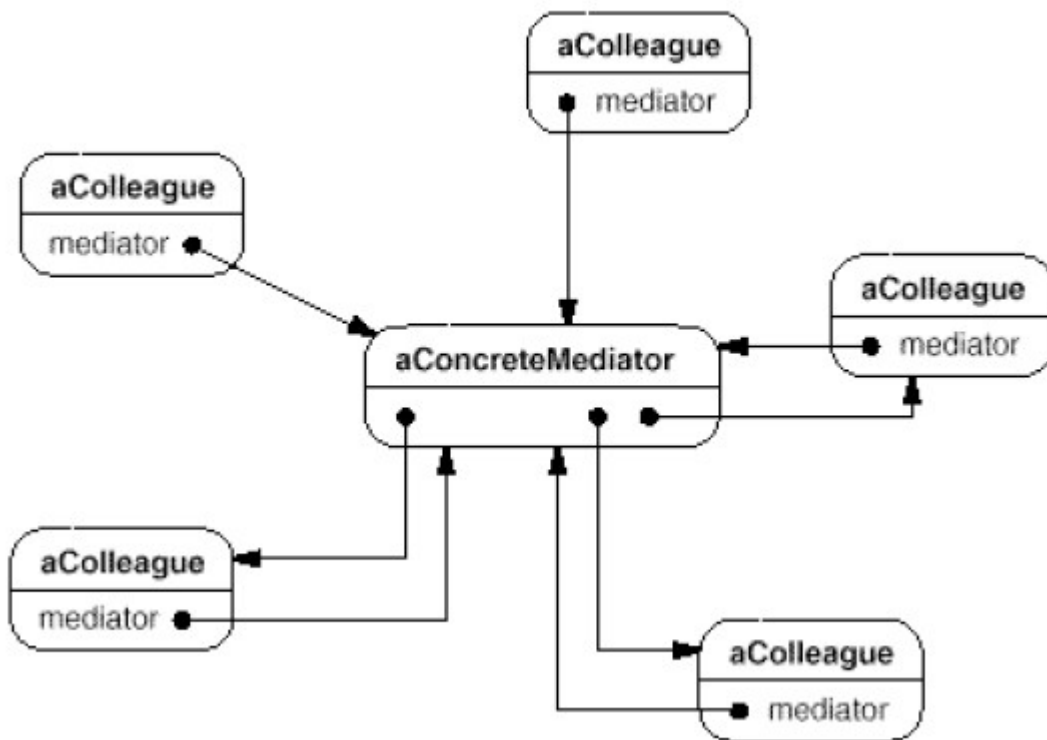
Astrae “il come” gli oggetti interagiscono permettendo di concentrarsi di più sull'interazione degli oggetti nel sistema.

Infine, concentra il controllo nel Mediator, sostituendo interazioni complesse con Mediator complesso, alle volte anche difficile ma mantenere.

## Implementazione:

La classe Mediator non serve che si astratta.

La comunicazione degli oggetti deve avvenire con il mediator quando si verifica un evento, a questo proposito il Mediator sarà implementato come un Observer, con tanto di metodi per la notifica.



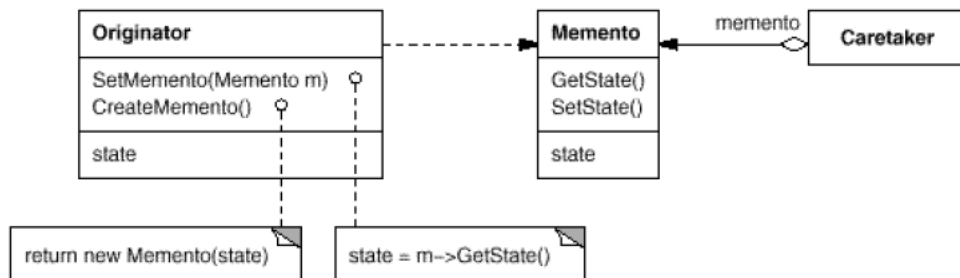
## Pattern Correlati:

1. Façade
2. Observer

## MEMENTO (RICORDO) [MMT]

- **Intento:** senza violare l'incapsulamento (dati nella parte Private), la cattura e l'esternalizzare lo stato interno di un oggetto allora che l'oggetto può essere ripristinato a questo stato dopo.
- **Anche conosciuto come:** Token.
- **Applicabilità:** Usare il Memento quando:
  - **Un snapshot dello (alcune porzioni di ) stato di un oggetto che deve essere salvato allora che è possibile ripristinare lo stato precedente e un interfaccia diretta per ottenere lo stato potrebbe esporre i dettagli implementativi e violare l'incapsulamento.**

- **Struttura:**



- **Partecipanti:**

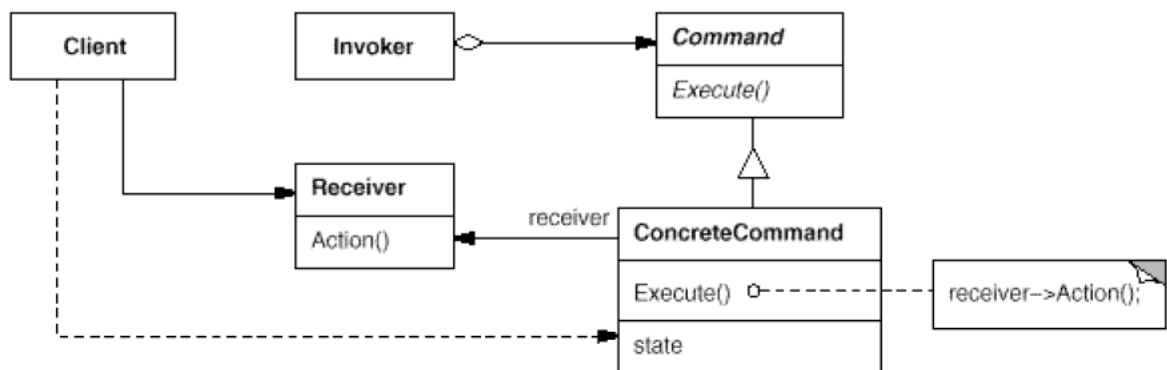
- **Memento:** conserva lo stato interno di un oggetto Originator. Potrebbe memorizzare tanto o poco dello stato interno dell'Originator. Protegge contro l'accesso da oggetti diversi dall'originator. Il Memento ha effettivamente due interfacce: 1. Il Caretaker vede un'interfaccia limitata per il memento. Può passare solo il memento ad altri oggetti. 2. L' Originator vede un'ampia interfaccia; accede a tutti i dati necessari per riportarsi allo stato precedente.
  - **Originator:** crea un memento che contiene uno snapshot del suo stato interno corrente. Usa il memento per ripristinare il suo stato interno.
  - **Caretaker (CUSTODE):** è responsabile di conservare i vari mementi. Non opera e non esamina il contesto del memento.
- **Collaborazioni:** Un caretaker richiede un memento da un originatore, lo trattiene per un po', e poi lo passa indietro all'originatore. Qualche volta il caretaker non passa il memento indietro all'originatore, poiché l'originatore potrebbe non avere bisogno di tornare ad uno stato precedente. I mementi sono passivi. Una volta che l'originatore ha creato il memento, gli assegnerà o riprenderà il suo stato. (passivi, xk i mementi non svolgono operazioni!!!)



- **Conseguenze:**
  - **Preservare i limiti dell'incapsulamento:** Il memento protegge gli altri oggetti dallo stato interno dell'originatore potenzialmente complesso questo stato, in tal modo preservando i limiti dell'incapsulamento.
  - **Il memento semplifica l'originatore:** i clienti gestiscono lo stato di cui hanno bisogno per semplificare l'originatore, e mantiene i clienti dal dover notificare l'originatore quando loro hanno finito di consultare lo stato.
  - **Utilizzare i mementi potrebbe essere costoso:** i mementi potrebbero incorrere in un notevole sovraccarico se l'Originatore deve copiare grandi quantità di informazioni, oppure i clienti creano e restituiscono spesso mementi all'originatore. Ovviamente il pattern Memento è adeguato se l'incapsulamento e il ripristino dello stato dell'Originatore sono poco costosi, altrimenti non è un pattern adeguato.
  - **Definire interfacce strette e larghe.** Può essere difficile in alcuni linguaggi garantire che solo l'originator può accedere allo stato del Memento.
  - **Nascondere i costi del memento.** Il carataker è responsabile per l'eliminazione di Mementi. Quindi il custode deve essere leggero, in modo tale da non incorrere in grandi spese di memorizzazione quando il deve memorizzare mementi nuovi.
- **Pattern Relativi:** Command, Iterator.

## COMMAND (COMANDO) [CMD]

- **Intento:** Incapsulare una richiesta come un oggetto, consentendo così di parametrizzare i clienti con differenti richieste, code o richieste di log, e supportare l'annullabilità delle operazioni.
- **Anche conosciuto come:** Azione, Transazione.
- **Applicabilità:** Usare il pattern Command quando voler:
  - **Parametrizzare l'oggetto per un'azione.**
  - **Specificare, code, ed eseguire richieste in tempi differenti.** Il CMD può avere un ciclo di vita indipendente dalla richiesta originale.
  - **Supporto undo:** Memorizzare lo stato per ritornare ad uno stato precedente, e tale operazione è costituita da un oggetto Command. Il CMD deve avere un'operazione aggiuntiva di Unexecute. Il Command eseguiti sono memorizzati in un lista (può ricordare il Memento?)
  - **Dare supporto per le modifiche di registrazione possono essere riapplicate in caso di un crash del sistema.** Si può ottenere, aggiungendo all'oggetto CMD le operazioni di load e store.
  - **Strutturare un sistema con operazioni di alto livello costruite su operazioni primitive.** Il CMD offre un modo per il modello di transazioni. Facilità per estendere il sistema in una nuova transazione.
- **Struttura:**



- **Partecipanti:**
  - **Command**: dichiara un'interfaccia per eseguire un'operazione.
  - **ConcreteCommand**: definisce un'associazione tra un oggetto **Receiver** e un'azione. Implementare "Execute" per invocare il corrispondente operazione sul **Receiver**.
  - **Client**: crea un oggetto **ConcreteCommand** e modifica il receiver.
  - **Invoker**: chiede ad un **Command** di restituire una richiesta.
  - **Receiver**: conoscere come eseguire l'operazione associata alla richiesta.

- **Collaborazioni:** il cliente crea un oggetto ConcreteCommand e specifica il suo receiver. Un oggetto Invoker memorizza l'oggetto ConcreteCommand. L'Invoker invia una richiesta chiamando il metodo Execute sul command. Quando il command è "undoable", il ConcreteCommand memorizza lo stato per ritornare allo stato precedente e quindi per annullare il comando prima di richiamare Execute. L'oggetto ConcreteCommand invoca l'operazione sul suo Receiver per restituire la richiesta.
- **Conseguenze:** Disaccoppiare l'oggetto che invoca l'operazione da un oggetto che invece conosce come svolgere l'operazione. I Command sono oggetti come tutti gli altri, ovvero possono essere manipolati ed estesi come qualunque oggetto. Si possono mettere insieme il command con composito di command. È facile aggiungere nuovi command, perché non devi cambiare le classi esistenti, ma aggiungere nuove classi command che sanno come svolgere una nuova operazione.
- **Pattern Relativi:** Composite, perché posso usare l'implementazione di MacroCommands. Memento, perché posso conservare lo stato del command richiesto in un undo. Il Command che deve essere copiato prima, inizia ad essere messo dentro la lista dei command utilizzati, che la lista utilizzerà tali command più comodamente come un Prototype.

## ***OBSERVER PATTERN***

L'intento del pattern è quello di definire una dipendenza 1-n fra i vari oggetti, cosicché quando un oggetto cambia il suo stato, tutte i suoi oggetti dipendenti sono notificati e aggiornati automaticamente.

Gli oggetti chiave del pattern sono due: il Soggetto & l'Osservatore. (Publisher & Subscribers)

Il Soggetto può avere qualsiasi numero di Osservatori da lui dipendenti.

Tutti gli Osservatori, invece, sono notificati ogni qual volta il Soggetto effettua un cambiamento di stato.

Il Pattern viene usato principalmente quando si verifica una delle seguenti condizioni:

- Un'astrazione possiede due aspetti differenti, una dipendente strettamente dall'altra;
- Un cambio di uno degli oggetti richiede il cambiamento degli altri oggetti (numero sconosciuto);
- Un oggetto notifica gli altri oggetti senza sapere nulla riguardo chi siano tali oggetti.

Il Soggetto conosce i suoi "n" Osservatori e provvede, quindi, a fornire un'interfaccia per aggiungere e/o rimuovere tali Osservatori.

L'Osservatore, invece, definisce un'interfaccia di aggiornamento per gli oggetti che dovrebbero essere notificati riguardo qualsiasi cambio avvenga nel Soggetto.

Il "Soggetto Concreto" immagazzina al suo interno lo stato di interesse agli Osservatori Concreti.

Gli "Osservatori Concreti" mantengono una referenza al Soggetto Concreto e implementano l'interfaccia di aggiornamento dell'Osservatore per mantenere aggiornato (consistente) lo stato del Soggetto.

Questo pattern, quindi, permette il riuso dei Soggetti senza riusare i loro Osservatori e viceversa; permette, inoltre, di aggiungere Osservatori senza modificare il Soggetto in sé o altri Osservatori a esso collegati.

Assicura un accoppiamento astratto fra il Soggetto e l'Osservatore: il Soggetto infatti possiede una lista dei suoi Osservatori ma non è a conoscenza di quale tipo reale essi siano. Tale disaccoppiamento garantisce che il Soggetto e gli Osservatori possano anche appartenere a due differenti livelli dell'applicazione (il Soggetto = basso livello, informa gli Osservatori = alto livello).

Conseguenze dell'applicazione del pattern sono, inoltre: la libertà di aggiungere e rimuovere in qualsiasi momento gli osservatori e la causa di update inaspettati; un'innocua operazione di update infatti potrebbe causare un'enorme cascata di chiamate di aggiornamento sugli Osservatori.

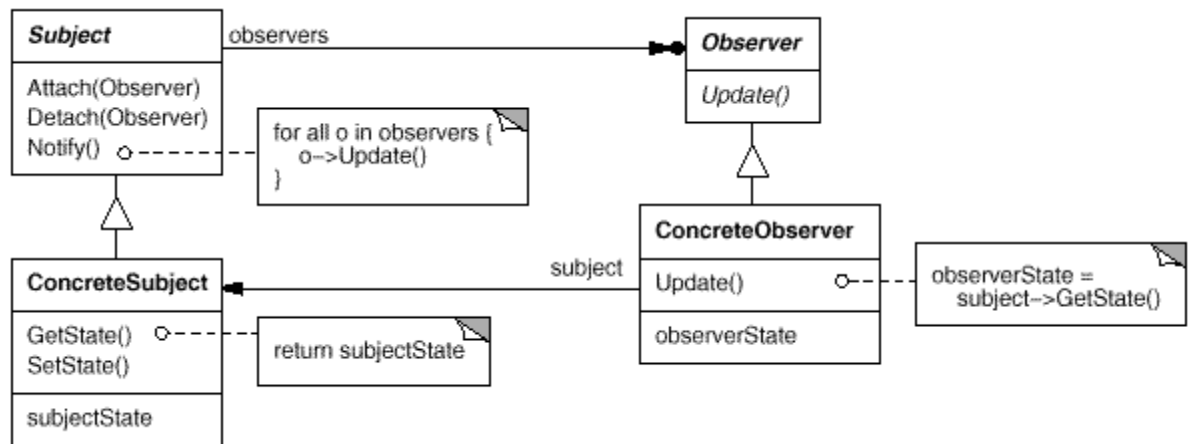
L'operazione di "Notifica" può essere a carico o dei metodi di SET o dei clients:

- Setters: i clients non devono preoccuparsi di chiamare il "notify", in quanto quest'ultimo viene chiamato automaticamente dopo il cambiamento di stato del Soggetto; può causare inefficienza a causa delle ripetute chiamate dei metodi di aggiornamento degli Osservatori.
- Clients: diventano responsabili per la chiamata del metodo "notify", permettendo di evitare chiamate ai metodi di aggiornamento inutilmente ed effettuare un singolo aggiornamento dopo una serie di cambiamenti di stato del Soggetto.

Il Soggetto, inoltre, deve occuparsi di notificare tutti i suoi Osservatori circa la sua eliminazione, così da chiamare i metodi di aggiornamento solamente nei casi in cui il Soggetto esiste realmente.

L' "Observer Pattern" può seguire due protocolli principali:

- 1) Pull: il Soggetto invia solamente una notifica minima ai suoi Osservatori, i quali chiederanno successivamente più informazioni circa lo stato aggiornato del Soggetto. Potrebbe risultare inefficiente poiché gli Osservatori devono accertarsi di cosa sia cambiato senza un aiuto reale da parte del Soggetto.
- 2) Push: il Soggetto invia ai suoi Osservatori informazioni dettagliate circa il suo cambiamento di stato.



Pattern Correlati:

- Mediator
- Singleton

## ***STATE PATTERN***

L'intento del pattern è quello di permettere ad un oggetto di alterare il suo comportamento quando si verifica un cambiamento del suo stato interno. L'oggetto in questione, infatti, apparirà come se avesse cambiato classe.

La classe Stato dichiara un'interfaccia comune a tutte le classi che rappresentano i diversi stati operazionali dell'Oggetto. Le sotto-classi della classe Stato devono implementare i comportamenti specifici dei vari stati.

Il pattern viene applicato in uno dei seguenti casi:

- Il comportamento di un particolare oggetto dipende dal suo stato ed esso deve cambiare il suo comportamento a tempo di esecuzione dipendentemente allo stato in cui si trova.
- Le operazioni hanno un grande blocco condizionale incentrato sullo stato dell'oggetto, e ognuno di questi stati dovrebbe essere trattato come un oggetto proprio che varia in maniera indipendente dagli altri oggetti.

I partecipanti coinvolti in questo pattern sono:

- 1) **Contesto:** definisce un'interfaccia di interesse per i clients e mantiene un'istanza di uno Stato Concreto che definisce lo stato corrente.
- 2) **Stato:** definisce un'interfaccia per incapsulare il comportamento associato con un particolare stato del Contesto.
- 3) **Stato Concreto:** ogni sotto-classe che implementa un particolare comportamento associato a un particolare stato in cui si trova il Contesto.

Il Contesto delega le richieste di uno specifico comportamento al corrente Stato Concreto.

Un Contesto potrebbe passare sé steso come argomento all'oggetto Stato che sta gestendo la richiesta, così da permettere all'oggetto Stato di accedere ai dati del Contesto.

Sia il Contesto che gli Stati Concreti possono decidere quale stato è il successivo e sotto quali circostanze questo nuovo stato prenderà il posto del vecchio.

Il pattern State mette tutto il comportamento associato con un particolare stato all'interno di un solo oggetto: nuovi stati e transizioni di stati possono essere aggiunti facilmente grazie alla definizione e all'aggiunta di nuove sotto-classi.

Lo State permette, inoltre, di evitare l'uso di grandi blocchi condizionali, così da rendere il codice più estensibile e soprattutto più semplice da interpretare.

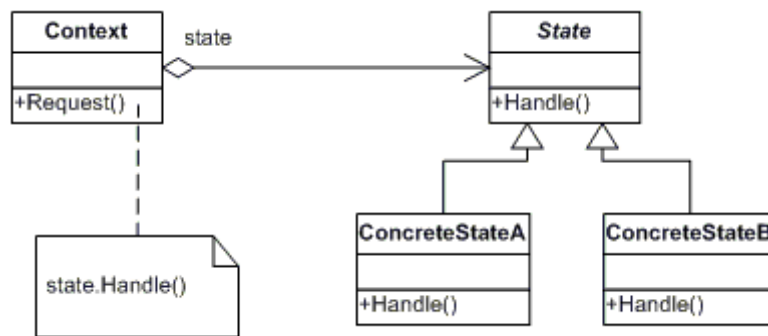
L'utilizzo di tale pattern permette, quindi, di rendere esplicite le transizioni fra i vari Stati, rappresentati come oggetti diversi e separati fra loro.

L'oggetto Stato, inoltre, può essere condiviso: se gli oggetti Stato, infatti, non posseggono stato intrinseco ma solo del comportamento, possono essere condivisi tramite l'applicazione del pattern Flyweight.

I passaggi tra i vari Stati, come detto, possono essere implementati anche interamente nel Contesto, con le conseguenti cause:

- 1) le classi Stato Concreto specificano il loro successore e il momento in cui effettuare la transizione
- 2) obbligo di aggiungere un'interfaccia al Contesto per permettere agli oggetti Stato di impostare il nuovo stato corrente del Contesto
- 3) svantaggio nel permettere agli Stati Concreti di conoscere almeno un altro Stato Concreto, introducendo così delle dipendenze di implementazione fra le varie sotto-classi.

Vantaggi nel creare gli oggetti Stato e non distruggerli mai è quello di avere un solo costo di istanziazione iniziale, nessun costo di distruzione e un minor overhead nel caso in cui i cambiamenti di stato avvengono abbastanza frequentemente. Creare e distruggere gli oggetti Stato, invece, risulta vantaggioso nel caso in cui gli stati cui si andrà incontro non sono conosciuti a priori e il cambiamento di stato avviene raramente.



Pattern Correlati:

- Flyweight
- Singleton

# Strategy Pattern

## Intento:

Definita una famiglia di algoritmi, li si incapsula singolarmente in classi, le quali potranno essere interscambiate.

Nella pratica questo pattern permette di variare l'algoritmo indipendentemente dal client utilizzatore.

## Motivazioni:

Bisogna scegliere al run-time la strategia/algoritmo da usare, e il metodo di selezione non è predicibile a priori.

## Quando applicare questo pattern ?

STG consente ad una classe di adottare il comportamento di una specifica classe, la quale appartenente ad un gruppo di altre classi correlate che differiscono soltanto dal comportamento.

Quando un algoritmo utilizza dati e strutture dati sconosciute al client.

Quando una classe definisce diversi comportamenti in relazione a dei costrutti condizionali, allora questi saranno spostati nella relativa classe strategica.

## Conseguenze:

Si svilupperà una famiglia di algoritmi riutilizzabili in base al contesto.

STG è un'alternativa al subclassing, consentendo di variare gli algoritmi dinamicamente, ed in modo completamente indipendente dal contesto rendendo più facile la comprensione e lo sviluppo.

Elimina i costrutti condizionali.

Permette di scegliere una diversa implementazione dello stesso comportamento.

Tra le Strategy e il Contesto ci sarà un pesante passaggio di dati/messaggi.

Incrementerà il numero delle classi e degli oggetti.

## Implementazione:

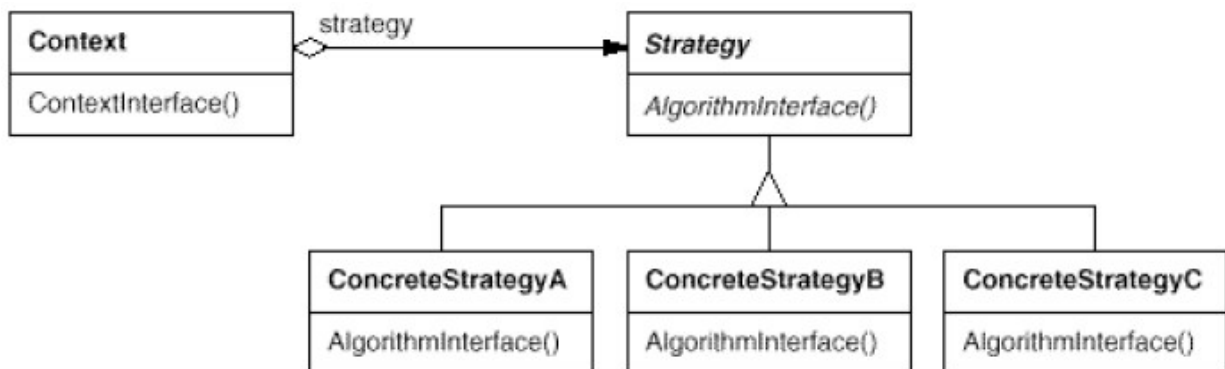
Bisogna definire le interfacce per le strategie e per il contesto.

L'implementazione può avvenire in più modi:

1. Il Contesto passa i dati alla Strategia come argomenti
  1. Il Contesto potrebbe passare dati superflui, ma tra Contesto e Strategia c'è disaccoppiamento.



2. Il Contesto si passa come argomento
  1. Il Contesto dovrà definire un'intefaccia d'accesso ai dati più complessi
  2. Strategie e Contesto saranno accoppiati, ma i dati richiesti dalla Strategia sono solo quelli necessari.
3. La Strategia sarà il parametro della classe template Contesto
  1. Lo Static Binding aumenta l'efficienza
4. Il Contesto controllerà la presenza di strategie, in caso negativo utilizzerà un comportamento di default.



# Template Method

## Intento:

Questo Pattern, definisce delle operazioni per un algoritmo, delega la concreta implementazione di alcune di queste alle sue sotto-classi, senza modificare la struttura dell'algoritmo stesso.

Nella classe Padre sono definiti i metodi astratti delle operazioni da ridefinire.

## Quando applicare questo pattern ?

Quando di un algoritmo alcune operazioni sono comuni e quindi saranno implementate nella classe padre, le altre invece saranno implementate nelle sotto-classi.

Quando si vuole eliminare codice duplicato spostandolo in una classe generale padre (refactoring to generalize) .

Quando si vuole definire un metodo template che richiamerà delle operazioni "hook", limitando l'estensione/implementazione nelle sotto-classi soltanto a queste operazioni.

## Conseguenze:

Il TM è fondamentale per il riuso del codice in quanto permette la creazione di librerie per l'implementazione delle operazioni astratte.

L'unico compito della classe template padre sarà richiamare queste operazioni "hook" e altri metodi utili alle sotto-classi.

Bisogna definire esattamente quali operazioni sono progettate per overriding, nella pratica:

- le operazioni "hook" bisognerebbe re-implementarle
- le operazioni astratte devono essere re-implementate

Per fare questo bisognerebbe utilizzare una convenzione sui nomi

## Implementazione:

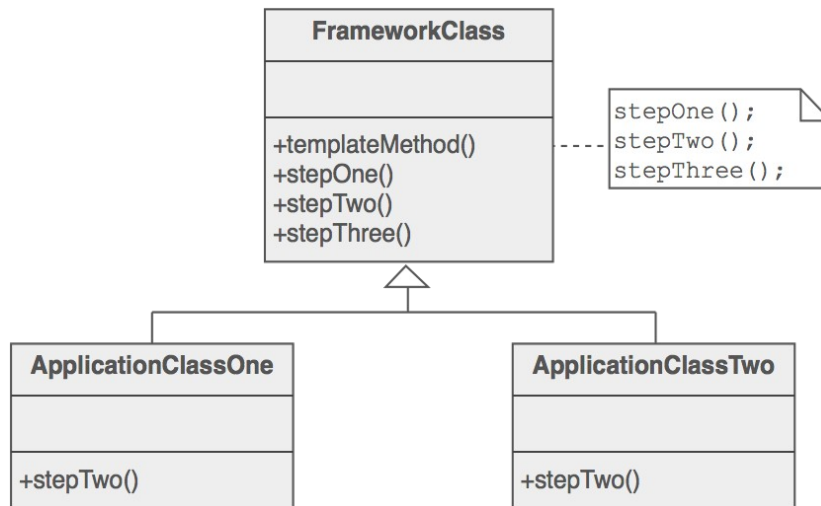
Le operazioni che il metodo template deve chiamare possono essere dichiarate protected, e quelle che devono essere re-implementate sono dichiarate virtuali pure.

Il metodo template non deve essere virtual.

Bisogna minimizzare le operazioni da re-implementare, adottando una convenzione sui nomi.

## Pattern correlati:

1. Metodo Fabbrica
2. Strategy



## **TEMPLATE**

Questo pattern, date e definite determinate operazioni per un algoritmo.

E' un pattern utile da applicare, quando per un algoritmo abbiamo operazioni che saranno comuni, e quindi sarà conveniente implementarle soltanto nella classe padre, lasciando alle sottoclassi la ridefinizione delle operazioni secondo le proprie necessità, che non sono comuni a tutte le altre, senza tuttavia modificare la struttura dell'algoritmo definito nella classe padre.

E' utile da utilizzare anche quando abbiamo la necessità di eliminare o non replicare codice, spostandolo soltanto all'interno della classe padre, e riferendosi soltanto ad esso.

Questo Pattern permette anche la creazione di vere e proprie librerie, in quanto avremo la possibilità di ridefinire le operazioni dell'algoritmo.

La classe padre template avrà quindi il solo compito di richiamare operazioni e metodi ridefiniti dalle sottoclassi.

Tuttavia soltanto due tipi di operazioni sono progettate per l'overriding:

- Le Operazioni **"HOOK"**, limitano l'estensione di queste operazioni alle sottoclassi, e bisognerebbe re-implementarle.
- Le Operazioni **"ASTRATTE"** dovranno essere re-implementate.

Le operazioni che il metodo template deve chiamare potranno essere dichiarate protected, e quelle da dover re-implementare adotteranno convenzioni sui nomi per esempio iniziando con do.

Pattern Correlati al metodo Template sono:

Strategy: in quanto le operazioni potranno essere implementate da più sottoclassi adottando comportamenti diversi per ognuna di esse, e definendo così una vera e propria strategia di algoritmi.

Metodo Fabbrica: Perché ridefinire un'operazione ricorda il FM, oppure perché potremmo avere una fabbrica di operazioni.

## INTERPRETER.

Il pattern Interpreter, si prefissa come obbiettivo, il comprendimento di una frase, di un determinato linguaggio, definita da alcune regole grammaticali.

Nel pattern interpreter, gli attori principali saranno:

Linguaggio: Concezione astratta (ex. Italiano)

Grammatica: (Regole definite sul Linguaggio)

Frase da Interpretare: (Contesto)

Lo usiamo quindi quando abbiamo la necessità di comprendere un linguaggio, e le regole della grammatica saranno rappresentate da una struttura ad albero.

Il pattern risulta essere veloce ed efficiente, se le regole della grammatica sono semplici e ben definite, in quanto se fossero complesse la gestione dell'albero potrebbe essere molto costosa.

Il contesto, ovvero l'oggetto che rappresenta la frase da interpretare, sarà passata all'interprete che avrà appunto il compito di comprenderla.

Sarà facile cambiare la sintassi se si utilizza una classe per rappresentare la grammatica.

A livello computazionale potrebbe essere molto complesso, gestire tante regole, in quando ognuna di esse rappresenta il nodo di un albero.

Le espressioni potranno essere:

Terminali: Se l'operazione ha un'interpretazione immediata

Non Terminale: Se l'operazione non ha un'interpretazione immediata riguarda una regola complessa e dovrà essere richiamata più volte (in modo ricorsivo) per essere calcolata, con l'ausilio anche delle espressioni terminali.

Pattern Correlati:

Composite:

Iterator: Perché possiamo iterare sulla struttura ad albero della grammatica.

FlyWeight: in quanto i Simboli del linguaggio se si ripetono più volte potranno essere memorizzati nei fly weight.

Visitor: Perché possiamo visitare la struttura dati.

## **MEDIATOR**

Fa da tramite, definendo un oggetto che incapsula un insieme di oggetti, per gestirne le iterazioni tra di loro, evitando che in questo insieme di oggetti ognuno di loro si riferisca in modo esplicito a uno o più oggetti dell'insieme.

Il Mediatore risolve quindi il problema che tutti gli oggetti si debbano conoscere, passando così da una dipendenza N: N ad una 1: N dove 1 è il Mediatore il quale:

- Si mette in mezzo a tutti gli oggetti dell'insieme.
- Controllerà e coordinerà le operazioni degli oggetti dell'insieme.
- Sarà lui a comunicare agli oggetti interessati, i cambiamenti degli oggetti che sono stati soggetti a modifiche, e di cui gli altri devono esserne a conoscenza.
- Gli oggetti dell'insieme manterranno un riferimento soltanto all'oggetto Mediatore, in quanto comunicheranno soltanto con lui, eliminando così il numero di interconnessioni tra oggetti.

Non innescheremo catene di modifiche, ma sarà soltanto il mediatore a comunicare i cambiamenti agli altri oggetti.

Lo utilizziamo quando abbiamo un insieme di oggetti, che comunicano in modo definito tra di loro, ma in modo complesso e vi è una forte interdipendenza, che vogliamo spezzare utilizzando il mediatore.

Utilizzabile anche quando vogliamo eliminare varie inconsistenze nel codice dovute alla modifica eliminazione o aggiunta di oggetti, poiché tra le interconnessioni degli oggetti, i cambiamenti saranno in più parti del codice, utilizzando il mediatore saranno concentrati soltanto al suo interno

Disaccoppieremo gli oggetti, la modifica di uno di essi non implicherà necessariamente quella di tutti gli altri.

Inoltre astrarremo il modo in cui gli oggetti comunicheranno tra di loro.

Pattern Correlati:

Observer: In quanto il mediatore può essere un osservato.

## **CHAIN OF RESPONSABILITY.**

L'idea della CoR è stata concepita quando ancora non vi era il meccanismo di gestione delle eccezioni, utilizzando sostanzialmente la stessa idea.

Lo utilizziamo per gestire la comunicazione, e la risoluzione di richieste tra più oggetti. L'obiettivo del pattern è di disaccoppiare la comunicazione tra più oggetti derivanti da varie problematiche.

- La comunicazione potrebbe non essere diretta, e passare per più oggetti.
- Prevedere in ogni oggetto i collegamenti verso altri oggetti, risulterebbe poco efficiente.

La CoR evita di accoppiare chi invia una richiesta, da chi la debba gestire, dando a più di un oggetto la chance di poterla gestire.

Una CoR di oggetti, si passa le richieste, il primo oggetto in grado di gestirla, la gestisce altrimenti la passa avanti. (Potrebbe accadere anche che vi sia un caso, che la richiesta non sia gestita da nessuno)

All'interno di un oggetto avremo un puntatore ad un altro oggetto della catena, che nel caso in cui, non riesca a gestire la richiesta, la chiamerà attraverso l'altro oggetto su cui invocherà il metodo per poterla risolvere.

## ***MEMENTO.***

È un ricordo, che sarà utilizzato senza violare l'incapsulamento dei dati ("Information Hiding")

Cattura lo stato interno di un oggetto in modo tale che possa essere ripristinato allo stato precedente in cui è stato salvato in un momento successivo, come se facessimo l'operazione di UNDO o (Ctrl + z)  
La fase di memorizzazione dello stato è il Memento stesso.

Il Memento sarà un Oggetto Opaco, in quanto colui che può riprenderlo è colui che lo ha creato e registrato.

Gli Attori del Memento saranno:

Originator: Sarà l'oggetto vero e proprio di cui voglio salvare lo stato, creerà il Memento e vi salverà il suo stato all'intero.

Memento: Conserva lo stato di un oggetto originator, salvandolo tutto o soltanto sue determinate parti.

Caretaker: E' il responsabile della gestione e conservazione dei memento, farà richieste prima di modificare l'oggetto originator, e quando vi è la necessità lo restituirà indietro.

Nessuno ci vieta di salvare il Memento e Riprenderlo anche dopo 10 giorni dovremmo serializzarlo e nel momento in cui dobbiamo riprenderlo, deserializzarlo.

Conseguenze:

- Potrà occupare molto spazio in memoria, in quanto l'originatore potrà copiare grandi quantità di dati all'interno del memento, oppure i client faranno richieste eccessive di memento, creandoli e salvandoli più volte.
- Nascondere i costi dei Memento, il caretaker dovrà occuparsi anche dell'eliminazione dei memento, in modo tale da non avere grandi spese di memorizzazione.

Pattern Correlati:

- Command, in quanto il Memento potrà essere un comando di Undo.
- Iterator, in quanto potremmo voler iterare sulla struttura dati che conserva i vari memento.



## COMMAND

Pattern molto utilizzato soprattutto per lo sviluppo di Gui, siti Web, o come controllore di architettura Model View Control.

Il Command incapsulerà, richieste come un oggetto, lasciando la possibilità di parametrizzare i client con diverse richieste, log, o richieste annullabili.

I CMD sono versioni ad oggetti che rimpiazzano gli EventHandler di java, utilizzando metodi ed oggetti. Torna utile e ci viene incontro per risolvere le problematiche dell'anti pattern (cuttenPestProgramming), non dovremmo più replicare codice e copiare codice in varie parti del programma che sono soggette alla stessa modifica, ma personificando i comandi all'interno di oggetti, le modifiche saranno effettuate soltanto all'interno dell'oggetto e non in tutte le varie parti del codice.

In ogni punto in cui prima avevamo codice uguale, adesso richiameremo soltanto il metodo dell'oggetto comando definito.

Il CMD:

- Parametrizza l'oggetto per un azione.
- Specifica code ed esegue richieste in tempi differenti, il ciclo di vita di un CMD sarà indipendente dalla richiesta.

Disaccoppieremo gli oggetti che invocano la richiesta da quelli che sanno come realizzarla e svolgerla. I Comandi essendo oggetti come tutti gli altri potranno essere manipolati ed estesi come qualsiasi altro oggetto.

Pattern Correlati:

- Memento: L'oggetto CMD potrà essere un Memento, se prevediamo un comando che debba ripristinare lo stato precedente di un oggetto applicandovi sopra un meccanismo di undo.
- Composite: Potremmo prevedere una struttura composta di comandi, in cui comando è composto da altri comandi al suo interno.
- FlyWeight: Se un comando svolge sempre le stesse funzioni, potremmo salvarlo all'interno di un oggetto flyweight è utilizzare soltanto quello.
- Prototype: Se ho la necessità di clonarli tutto dallo stesso punto.

## **VISITOR**

E' un visitatore, il suo intento è quello di rappresentare un operazione che dovrà essere eseguita sugli elementi di un oggetto struttura.

Il visitor ci consentirà di definire una nuova operazione senza cambiare le classi dell'oggetto struttura su cui opera.

Verrà utilizzato quando:

- Un oggetto struttura contiene diverse classi di oggetti, definite da una o più interfacce e vogliamo eseguire delle operazione che dipendono dagli oggetti definiti dalle classi concrete che implementano le interfacce dell'oggetto struttura.
- Se le classi che definiscono l'oggetto sono soggette a rari cambiamenti e vi vogliamo effettuare molto spesso differenti operazioni.

Il visitor potrà essere anche un interfaccia, e le sottoclassi che la implementano, implementeranno ciascun metodo di visita previsto dall'interfaccia, secondo le necessità di cui ha bisogno la sottoclasse.

Un client crea un visitor, e creando un oggetto visitor, riuscirà a visitare ogni elemento concreto dell'oggetto struttura.

Quando si visita un elemento, esso passerà se stesso al visitor, consentendogli di accedere al suo stato interno e quindi di poterlo visitare secondo l'operazione di visita definita nel visitor.

L'operazione dipenderà sia dal visitor che utilizziamo, sia dall'oggetto visitato. Una delle particolarità di questo pattern è l'utilizzo del metodo double-dispatch che si concretizza all'interno del metodo accettato dell'oggetto da visitare.

Pattern correlati:

- Composite: Possiamo effettuare una visita su un oggetto struttura, definito all'interno di una struttura composita.
- Interpreter: La visita potrebbe essere utilizzata per effettuare l'interpretazione del contesto.

## **STRATEGY**

È un pattern che possiamo applicare quando abbiamo definita una famiglia di algoritmi

Il suo intento è quello di incapsulare al suo interno singole classi, che potranno essere inter scambiate tra di loro.

Lo scopo principale del pattern è quello di variare l'algoritmo che si vuole utilizzare indipendentemente dal client utilizzatore.

Potremmo scegliere a run-time la strategia, o l'algoritmo da utilizzare.

La strategia quindi ci consentirà all'interno di una classe, di utilizzare e adottare al suo interno un algoritmo o un comportamento specifico di un'altra classe, che apparterrà ad un altro gruppo di classi.

Svilupperemo quindi una famiglia di algoritmi, che potremmo utilizzare in base al contesto in cui si opera, gli algoritmi quindi potranno variare indipendentemente dal contesto, in quanto potremmo scegliere diverse implementazioni per uno stesso comportamento. (Per esempio l'ordinamento di un vettore (comportamento), potremmo effettuarlo con diverse metodologie (bubble-sort, insertion-sort, ecc.... (che sono le implementazioni del comportamento.)).

Pattern Correlati.

## **DISCUSSIONI FINALI.**

Il mediatore gestisce il protocollo di iterazione tra oggetti.

L'iteratore invece gestisce il modo con cui attraversare gli oggetti.

Tipicamente molti pattern passano oggetti come argomenti, ciò accade nel visitor, nel command, nel memento (per ripristinare lo stato).

Comunicazione distribuita o incapsulata (Mediatore contro Observer)

Observer gestisce quella distribuita.

Mediatore quella incapsulata.

Disaccoppiare chi invia un messaggio da chi reagisce alla richiesta, possiamo farlo con vari pattern quali l'observer, il command, la CoR.

-CoR è una catena in cui ognuno conosce il successivo

-Mediatore i messaggi confluiscono tutti quanti in modo centralizzato verso di lui

-Observer fa una struttura, in cui ce un invio di messaggi e di notifiche ad eventi.

Combinazioni di pattern,

CoR usa il template perché ->

CoR usa il comando per rappresentare le richieste.

CoR si integra bene con il composito quando mettiamo il link al padre.

Interpreter può usare lo State per gestire l'interpretazione dei contesti,

Visitor può implementare le valutazioni di un interpreter.

Visitor si integra bene con il Composite.

## ***VISITOR PATTERN***

L'intento del Visitor è quello di rappresentare un'operazione che deve essere effettuata sugli elementi di un oggetto struttura.

Il Visitor lascia definire una nuova operazione senza cambiare le classi degli elementi su cui opera.

Il pattern viene utilizzato quando:

- 1) Un oggetto struttura contiene diverse classi di oggetti con differenti interfacce e si vogliono eseguire operazioni su questi oggetti che dipendono dalle classi concrete.
- 2) Molte operazioni distinte e quindi tra loro scollegati devono essere eseguite sugli oggetti in un oggetto struttura
- 3) Se le classi che definiscono l'oggetto struttura cambiano raramente e si vogliono definire spesso delle nuove operazioni sulla struttura; se la struttura delle classi cambia si devono definire nuove operazioni su queste classi.

Partecipanti a questo pattern sono:

- Visitor: dichiara un'operazione di "Visita" per ogni Elemento Concreto e può accedere direttamente all'elemento tramite la sua particolare interfaccia.
- Visitor Concreto: implementa ciascuna operazione dichiarata dal Visitor e fornisce un contesto per l'algoritmo.
- Elemento: definisce un'operazione "Accept", che prende in input un Visitor.
- Elemento Concreto: implementa l'operazione Accept che prende come argomento un'istanza di Visitor
- Oggetto Struttura (Programma): può enumerare i suoi elementi e può fornire un'interfaccia ad alto livello per permettere al Visitor di visitare i suoi elementi; può essere o un oggetto composito o una collezione, come una lista o un insieme.

Ciò che avviene utilizzando tale pattern è il seguente:

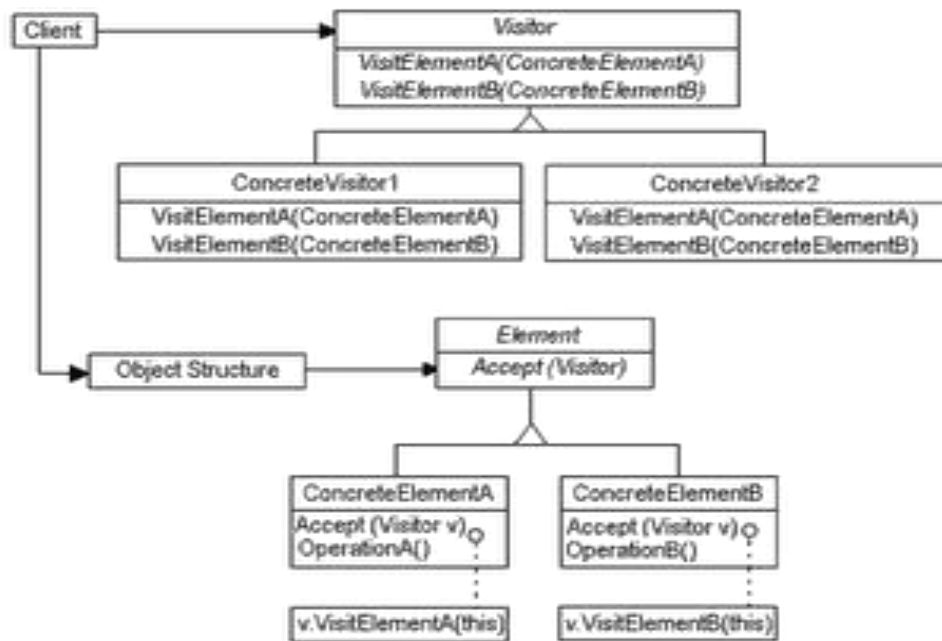
- Un client crea un Visitor Concreto che attraversa l'oggetto struttura visitando ogni suo elemento. Quando si visita uno degli elementi si chiama l'operazione del Visitor che corrisponde alla gestione di tale oggetto; l'elemento stesso, infatti, passa sé stesso al Visitor così da permettere ad esso di accedere al suo stato interno, se necessario durante la visita.

Ogni oggetto struttura avrà una classe Visitor associata ad esso. Questa classe astratta "Visitor" avrà un'operazione di visita per ogni elemento concreto all'interno della struttura da visitare.

Ogni operazione di visita ha, quindi, come argomento un elemento concreto da visitare.

Le classi concrete del "Visitor" implementeranno, quindi, ciascun metodo visita della classe astratta, così da permettere per ogni sotto-classe un metodo differente di visita della struttura.

L'operazione effettuata, quindi, dipende sì dal tipo del particolare Visitor utilizzato ma anche dall'oggetto visitato. Particolarità di questo pattern è, infatti, il cosiddetto "double-dispatch", che si concretizza nel metodo ACCEPT dell'oggetto da visitare.



Pattern Correlati:

- Composito: si può effettuare una visita su un oggetto struttura definito da una composizione
- Interprete: la visita potrebbe essere impiegata per effettuare l'interpretazione

## Pattern Creazionali.

**Fabbrica:** Abstract Factory, Prototype.

**Fabbrica Astratta:** Factory method, Prototype, Singleton.

**Singleton:** Abstract Factory, Builder, Prototype.

**Prototype:** AF, Composite, Decorator.

**Builder:** Abstract Factory, Composite.

## Pattern Strutturali.

**Adapter:** Bridge, Decorator, Proxy.

**Bridge:** Abstract Factory, Adapter.

**Composite:** Iterator, Flyweight, Decorator, Visitor.

**Decorator:** Adapter, Composite, Strategy.

**Facade:** Abstract Factory, Mediator, Singleton.

**Flyweight:** Composite, State, Strategy.

**Proxy:** Adapter, Decorator.

## Pattern Comportamentali.

**Catena di Responsabilità:** Composite.

**Interprete:** Composite, Iterator, Flyweight, Visitor.

**Mediator:** Facade, Observer.

**Memento:** Command, Iterator.

**Observer:** Mediator, Singleton.

**State:** Flyweight, Singleton.

**Strategy:** Flyweight.

**Template:** Factory, Strategy.

**Visitor:** Composito, Interprete.

**Command:** Memento, Composite, Flyweight, Prototype.

**Iterator:** Composite, Factory, Memento.

## Analisi dei requisiti.

L'analisi dei requisiti è un'attività preliminare allo sviluppo, o alla modifica, di un sistema software con lo scopo di definire le funzionalità e gli obiettivi che il nuovo sistema deve fornire.

L'analisi dei requisiti è una fase presente in tutti i modelli di ciclo di vita del software, pur con diverse enfasi e diverse connotazioni.

Nel modello a cascata, l'analisi dei requisiti è la prima fase del processo di sviluppo, e deve concludersi con la stesura di una dettagliata **specificazione dei requisiti** che descrive le funzionalità del nuovo software nella loro interezza; tale specifica guida le fasi successive di sviluppo, che complessivamente sono volte a realizzare quanto previsto da tale specifica.<sup>[2]</sup>

L'analisi dei requisiti avviene normalmente come negoziazione fra individui legati allo sviluppo (analisti) e i clienti, fra analisti e responsabili del marketing. Tale dialogo è tutt'altro che semplice: gli analisti possono avere difficoltà a comprendere il linguaggio e il contesto culturale del cliente, e viceversa; inoltre lo stesso cliente potrebbe aver difficoltà a mettere a fuoco i propri reali bisogni e di conseguenza le richieste o le proposte da mettere sul tavolo della discussione. Proprio a causa di queste difficoltà, i modelli di ciclo di vita del software moderni hanno abbandonato l'assunzione che sia possibile identificare i requisiti di un sistema software *a priori*, e tendono a privilegiare approcci iterativi in cui i requisiti vengono esplicitati gradualmente, per esempio coinvolgendo l'utente nella prova di prototipi e rilasci parziali del sistema in corso di sviluppo.

Il documento principale prodotto dall'analisi dei requisiti è la specifica dei requisiti; se la metodologia e il modello di ciclo di vita del software utilizzati lo prevedono, essa può addirittura portare già alla stesura del manuale d'uso del prodotto da sviluppare.

## Requisiti Utente.

Descrivono requisiti funzionali e non funzionali, espressi in modo da risultare comprensibili agli utenti del sistema sprovvisti di conoscenze tecniche. I requisiti utenti sono generalmente espressi in linguaggio naturale, tenendo in considerazione alcune linee guida:

- usare un formato standard per tutti i requisiti
- usare il linguaggio naturale in modo consistente  
(es. uso di "deve" per requisiti necessari e "dovrebbe" per requisiti desiderabili)
- evidenziare le parti fondamentali di un requisito
- evitare l'uso di termini tecnici

Sono le informazioni derivate dai colloqui con il cliente. In esso si deve segnare tutto ciò che il sistema dovrà essere in grado di fare per soddisfare il cliente.



## Requisiti di Sistema.

Sono specificati mediante la stesura di un documento strutturato che descrive in modo dettagliato i servizi che il sistema software deve fornire.

Il documento risultante costituisce un "contratto" tra cliente e fornitore.

Schede Casi d'Uso.

Il documento dei casi d'uso individua e descrive gli scenari elementari di utilizzo del sistema da parte degli attori che si interfacciano con esso. \*

Diagramma dei Casi d'uso.

È una rappresentazione grafica dei casi d'uso, e degli attori dei casi d'uso stessi, rappresentandone le relazioni tra essi. \*

Diagramma di sequenza (SSD).

Un SSD è una figura che mostra, per un preciso scenario di un caso d'uso, gli eventi generati dagli attori esterni, le risposte del sistema e il loro ordine.

Gli eventi di input che provocano cambiamenti nel sistema sono detti **eventi di sistema**. Un SSD mostra gli eventi di sistema per un solo scenario di un caso d'uso è dunque generato per ispezione da un caso d'uso.

\*

Diagramma di dominio.

Indica una rappresentazione di classi concettuali del mondo reale, non di oggetti software.

Non indica un insieme di diagrammi che indicano classi software, lo strato del dominio di un'architettura software, lo strato del dominio di un'architettura software o oggetti software con responsabilità. \*

Contratti.

I contratti delle operazioni possono essere definiti per le operazioni di sistema. Le operazioni di sistema possono essere identificate attraverso gli SSD. \*

Architettura.

Un'architettura è l'insieme delle decisioni significative sull'organizzazione di un sistema software, la scelta degli elementi strutturali da cui è composto il sistema delle relative interfacce, insieme al loro comportamento specificato dalle collaborazioni tra questi elementi, la composizione di questi elementi strutturali e comportamentali in sottosistemi via via più ampi, e lo stile architetturale che guida questa organizzazione. L'architettura logica è l'organizzazione su larga scala delle classi software in package, sottosistemi e strati. Uno strato è un gruppo di package, sottosistemi o classi, che ha responsabilità coese a un aspetto importante del sistema. Gli strati sono organizzati in modo che gli strati "più alti" ricorrano ai servizi degli strati "più bassi", mentre, di solito, non è possibile il contrario.

Normalmente i strati che compongono un architettura sono:

- User Interface
- Application Logic e Domain Object
- Technical Services.

\* VEDERE CONSEGNA DOCUMENTAZIONE PROGETTO PER ESEMPLI.

## PATTERN CREAZIONALI.

### METODO FABBRICA:

- Abstract Factory: i metodi fabbrica sono spesso utilizzati per l'implementazione della fabbrica astratta.

### FABBRICA ASTRATTA:

- Factory method : la fabbrica astratta viene spesso implementata con l'utilizzo di metodi fabbrica.
- Prototype : si possono creare i prototipi di ciascun prodotto e creare gli altri tramite la clonazione.
- Singleton : una fabbrica astratta può essere implementata con il singleton in modo da avere una sua singola istanza.

### SINGLETON:

Abstract Factory, Builder, Prototype : è molto utile per questi pattern avere una sola istanza da utilizzare.

### PROTOTYPE:

- Abstract Factory e Builder: la fabbrica astratta potrebbe memorizzare un insieme di prototipi usati come punto di partenza, da clonare per restituire gli oggetti prodotto. Come i *pattern* Abstract factory e Builder, - Prototype: permette di incapsulare al suo interno la modalità di istanziazione degli oggetti, liberando i client dalla necessità di conoscere i nomi delle classi da istanziare.
- Composite e Decorator: entrambi possono trarre vantaggio dall'utilizzo del pattern Prototype.

### BUILDER:

- Abstract Factory : È simile al Builder perché anche esso consente la creazione di oggetti complessi; la differenza è che il Builder è incentrato sulla costruzione passo dopo passo, restituendolo solo il risultato finale.
- Composite : Il Builder spesso viene utilizzato per costruire un Composite.

## PATTERN STRUTTURALI

### ADAPTER:

- Bridge : Bridge e Adapter hanno una struttura simile, ma il Bridge separa un'interfaccia dalla sua implementazione.
- Decorator e Proxy.

Questo *design pattern* è parte fondamentale della programmazione a oggetti ed è stato formalizzato per la prima volta da Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides - la cosiddetta gang of four - nel libro Design Patterns (non c'entra un cazzo, ma la gang of four è troppo cool :D).

### BRIDGE:

- Abstract Factory : può essere utilizzata per creare e configurare un Bridge
- Adapter : l'adapter fa collaborare tra di loro classi non correlate e viene solitamente introdotto dopo che queste sono state progettate. Il Bridge invece è spesso utilizzato all'inizio di un progetto per consentire ad astrazione ed implementazione di variare in modo indipendente.

### COMPOSITE:

- Catena di responsabilita: i riferimenti al componente padre sono spesso utilizzati nel Pattern CoR.
- Decorator: viene spesso usato con il Composite. Quando accade, solitamente hanno una superclasse comune.
- FlyWeight : consente di condividere i componenti pur tuttavia non mantenendo i riferimenti al padre. - Iterator : può essere utilizzato per la visita di strutture composite.

- Visitor : localizza operazioni e comportamento che altrimenti dovrebbero essere distribuiti fra le classi Leaf e Composite.

#### DECORATOR:

- Adapter: differiscono in quanto il Decorator modifica soltanto la responsabilità di un oggetto e non la sua interfaccia, mentre un adattatore fornisce a un'oggetto un'interfaccia completamente nuova.
- Composite : un decoratore può essere visto come un oggetto composito degenero costituito da un singolo componente. Ad ogni modo un decoratore è usato per aggiungere responsabilità ad un oggetto e non è pensato per gestire l'aggregazione tra oggetti.
- Strategy : un Decoratore consente di cambiare la "pelle" di un oggetto, mentre la Strategy consente di cambiarne "gli organi interni". Si tratta di due alternative utilizzabili per modificare il comportamento di un'oggetto.

#### FACADE:

- Abstract Factory : può essere utilizzato insieme al Facade per fornire un'interfaccia per la creazione di oggetti del sottosistema in maniera indipendente dallo specifico sottosistema considerato. Inoltre, può essere utilizzata in alternativa al Facade per nascondere classi specifiche di una particolare piattaforma.
- Mediator : perché è molto simile al Facade, visto che fornisce un'astrazione delle funzionalità delle classi esistenti: il suo obiettivo è quello di fornire un'astrazione di una generica comunicazione fra oggetti.
- Singleton : di solito, viene richiesta l'esistenza di un solo oggetto Facade.

#### FLYWEIGHT:

- Composite: il flyweight è spesso utilizzato insieme al Composite per implementare strutture gerarchiche.
- State.
- Strategy.

#### PROXY:

- Adapter : un adattatore fornisce un'interfaccia diversa per l'oggetto adattato, mentre un proxy fornisce la stessa interfaccia dell'oggetto rappresentato.
- Decorator : i decoratori possono avere implementazioni simili al Proxy, ma hanno un obiettivo differente: un decoratore aggiunge una o più responsabilità a un oggetto, mentre un proxy controlla l'accesso a un'oggetto.

## PATTERN COMPORTAMENTALI

#### CATENA DI RESPONSABILITA':

- Composite: la catena può essere vista come una struttura composita.

#### INTERPRETE:

- Composite : nell'interprete le varie regole sono presenti in nodi di un albero, il composito risulta molto utile per modellare la struttura.

- Iterator: con l'iterator è possibile muoversi all'interno della struttura.

- Flyweight:

- Visitor.

#### MEDIATORE:

- Facade.

- Observer.

## MEMENTO:

- Command: il command può essere salvato nell'UNDO di un momento.
- Iterator: con l'iterator si possono scorrere i mementi.

## OBSERVER:

- Mediator.
- Singleton: si può richiedere una sola istanza di observer.

## STATE:

- Flyweight.
- Singleton: lo state può essere implementato come singleton.

## STRATEGY:

- Flyweight.

## TEMPLATE:

- Metodo Fabbrica.
- Strategy.

## VISITOR:

- Composito: si può effettuare una visita su di un oggetto struttura definito da una composizione
- Interprete: la visita potrebbe essere impiegata per effettuare l'interpretazione

## COMMAND:

Memento: è possibile conservare lo stato del command richiesto in un UNDO

Composite .

Flyweight.

Prototype : si possono usare i prototipi per istanziare oggetti.

## ITERATOR:

- Composite, Memento: con l'iterator ci si può muovere all'interno delle strutture.
- Factory